arXiv:2603.23910v1 [cs.AI] 25 Mar 2026

AnalogAgent: Self-Improving Analog Circuit Design Automation with LLM Agents

Zhixuan Bao*
Nanyang Technological University
Singapore
zhixuan005@e.ntu.edu.sg

Zhuoyi Lin†
Institute for Infocomm Research
Singapore
Lin_Zhuoyi@a-star.edu.sg

Jiageng Wang
Nanyang Technological University
Singapore
WANG2396@e.ntu.edu.sg

Jinhai Hu
Institute of Microelectronics
Singapore
hujh@a-star.edu.sg

Yuan Gao
Institute of Microelectronics
Singapore
gaoy@a-star.edu.sg

Yaoxin Wu
Eindhoven University of Technology
Eindhoven, Netherlands
y.wu2@tue.nl

Xiaoli Li
Singapore University of Technology and Design, Singapore
xiaoli_li@sutd.edu.sg

Xun Xu
Institute for Infocomm Research
Singapore
Xu_Xun@a-star.edu.sg

Abstract

Recent advances in large language models (LLMs) suggest strong potential for automating analog circuit design. Yet most LLM-based approaches rely on a single-model loop of generation, diagnosis, and correction, which favors succinct summaries over domain-specific insight and suffers from context attrition that erases critical technical details.To address these limitations, we propose **AnalogAgent**, a training-free agentic framework that integrates an LLM-based multi-agent system (MAS) with self-evolving memory (SEM) for analog circuit design automation. AnalogAgent coordinates a Code Generator, Design Optimizer, and Knowledge Curator to distill execution feedback into an adaptive playbook in SEM and retrieve targeted guidance for subsequent generation, enabling cross-task transfer without additional expert feedback, databases, or libraries. Across established benchmarks, AnalogAgent achieves 92% Pass@1 with Gemini and 97.4% Pass@1 with GPT-5. Moreover, with compact models (e.g., Qwen-8B), it yields a +48.8% average Pass@1 gain across tasks and reaches 72.1% Pass@1 overall, indicating that AnalogAgent substantially strengthens open-weight models for high-quality analog circuit design automation.

Keywords

Analog Circuit Design, Large Language Model, Agentic AI.

*Work done during internship at A*STAR.
†Corresponding author.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
Conference acronym 'XX, Woodstock, NY
© 2018 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-1-4503-XXXX-X/2018/06
https://doi.org/XXXXXXX.XXXXXXX

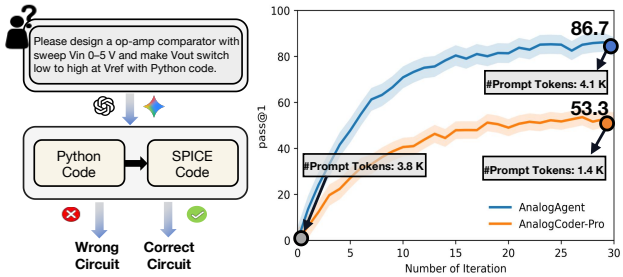


Figure 1: **Illustration of context attrition on the hard Task 25.** As context accumulates across iterations, LLM agents tend to compress it into shorter, less informative summaries, diminishing instruction salience and degrading detailed guidance. AnalogCoder Pro takes ~7 minutes to produce its first correct circuit, whereas AnalogAgent succeeds in 17.13 seconds with learned knowledge.

1 Introduction

Analog circuit design is a cornerstone of modern scientific discovery, enabling the reliable translation of physical phenomena into calibrated electrical signals that can be measured, controlled, and computed with high fidelity. It underpins core instrumentation across biomedical sensing, environmental monitoring, and large-scale experimental facilities, where noise, accuracy, and stability directly shape data quality and reproducibility [11]. In contrast to digital design, which benefits from standardized abstractions and mature automation pipelines, analog design relies heavily on expert knowledge, heuristic reasoning, and iterative simulation-driven refinement. Although conventional electronic design automation (EDA) tools have achieved progress in areas such as device sizing and topology optimization [4, 31], they are typically limited to predefined architectures [24], require extensive manual configuration [15], and scale poorly when exploring novel circuit structures or heterogeneous design objectives [20, 23].

Recent advancements in large language models (LLMs) have revitalized academic interest in analog circuit design automation. By leveraging their capabilities in code generation, structured reasoning, and broad generalization, LLMs offer an alternative to traditional workflows that rely primarily on hand-crafted rules or tightly coupled numerical optimization [12, 13, 30, 32]. Early evidence is provided by AnalogCoder, which shows the feasibility of generating executable design scripts for analog circuits in a code-centric pipeline [12]. AnalogCoder-Pro further introduces a multimodal feedback-enhanced design flow [13]. These efforts establish an important benchmark for LLM-driven analog design automation with training-free code generation. However, LLM-based analog circuit design automation remains constrained by several challenges. **(1) Single LLM paradigm.** Most approaches rely on a single LLM to jointly handle code generation, error diagnosis, and iterative refinement, coupling heterogeneous subtasks within one monolithic loop. **(2) Context attrition.** As shown in Figure 1, LLM agents tend to frequently rewrite the detailed specification and prior findings during iterative refinement, which progressively erode salient technical details and degrade the performance. **(3) Dependence on circuit libraries.** Most of the existing methods assume access to additional circuit databases (e.g., topology library, circuit textbooks, and literature) to scaffold circuit design, which limits applicability in settings where such resources are unavailable or proprietary.

In this paper, we introduce AnalogAgent, a training-free agentic framework for analog circuit design automation. Unlike prior LLM-based methods that center on a single agent augmented with circuit libraries, AnalogAgent integrates an LLM-based **multi-agent system (MAS)** with **self-evolving memory (SEM)** to adaptively execute design, evaluation, and knowledge reuse. Specifically, AnalogAgent coordinates three agents. (1) The Code Generator generates task-specific SPICE/Python design scripts. (2) The Design Optimizer evaluates candidates through simulation-driven checks and converts execution feedback into targeted corrections and reusable best practices. (3) The Knowledge Curator distills optimization feedback into structured, reusable design knowledge and incrementally updates the **Adaptive Design Playbook** within SEM. The playbook then supplies targeted guidance to the Code Generator, grounding subsequent generations in accumulated experience. By coupling workflow decomposition with iterative execution feedback and structured memory updates, AnalogAgent converts outcomes into actionable guidance, reduces redundant experimentation, and accelerates convergence to specification-compliant designs.

In summary, the main contributions of this work are: (1) **Conceptual**: AnalogAgent represents an early attempt at a training-free agentic framework for analog circuit design automation that operates without additional circuit libraries. (2) **Algorithmic:** AnalogAgent employs a coordinated multi-agent system with Code Generation, Design Optimization, and Knowledge Curation agents to support structured reasoning and sustained self-improvement. Its self-evolving memory mechanism retrieves guidance from an Adaptive Design Playbook and distills validated heuristics into reusable rules, mitigating context attrition and improving robustness and cross-task generalization. (3) **Empirical:** We evaluate AnalogAgent and strong baselines on analog circuit design tasks spanning multiple complexity levels. Results show consistent gains and successful adaptation to compact open-weight models (e.g., Qwen-8B).

2 Related Work

2.1 Circuit Design Automation

Conventional analog automation tools provide practical benefits, yet key constraints remain. Recent device-sizing methods improve sample efficiency [26, 31], with further gains from learned optimization and parallelization [3, 4]; related learning-based sizing approaches report similar efficiency improvements [7, 14], and newer systems advance sample-efficient optimization [22, 25]. Despite this progress, these methods are largely confined to predetermined circuit architectures, consistent with earlier pipelines [15], which limits transfer when meeting specifications requires architectural adaptation, alternative biasing, or changes in feedback structure; they work best when the topology class is known and the objective reduces to parameter selection within a fixed template. In contrast, topology exploration often incurs substantial simulation overhead [20, 23], depends on expert-derived analytical formulations [21, 29, 35], and offers limited structural diversity [19], while risking invalid configurations [18]. Costs are dominated by feasibility screening across many candidates, with frequent failures from bias infeasibility, missing DC paths, or non-convergent operating points; expert dependence further reduces portability across circuit families, regimes, or process assumptions. Moreover, automated synthesis approaches such as [24] often require substantial up-front setup, restricting deployment to predefined component or topology libraries, including block curation, interface/range specification, and constraint encoding for correctness and simulator stability, which raises the barrier to extending the system to new components, macro-blocks, or architectural motifs.

2.2 LLM-based Circuit Design Automation

Early LLM-based analog circuit design approaches largely rely on prompt-driven generation, using curated instructions and in-context exemplars to steer LLMs toward synthesizing SPICE or Python netlists, as demonstrated by SPICEPilot [30], Artisan [6], and Ado-llm [32]. A complementary paradigm explores training-based solusions, improving circuit synthesis via supervised fine-tuning, reinforcement learning, or dataset augmentation. Representative methods include AnalogGenie [10], LaMAGIC [5], and CKTGNN [8]. While effective, these approaches typically require additional labeled data and may demand repeated retraining when generalizing to new circuit families. To further enhance reliability and reduce iterative search cost, recent work introduces external knowledge resources such as topology libraries, circuit tool repositories, or structured databases to condition generation, including AnalogXpert [33], AMSnet-KG [27], LADAC [17], LayoutCopilot [16], AnalogCoder [12], and AnalogCoder-pro [13]. In contrast, AnalogAgent is training-free and does not rely on an additional database or a pre-built circuit library. Instead, it leverages execution signals from simulation and checker diagnostics, distilling both failures and successes into reusable rules that are stored in SEM and retrieved to guide subsequent generations, thereby forming a cross-task self-improvement loop. Moreover, AnalogAgent maintains strong performance with smaller models that are suitable for local or self-hosted deployment, reducing reliance on large proprietary backbones while retaining reliable first-pass synthesis and iterative refinement for industry companies.

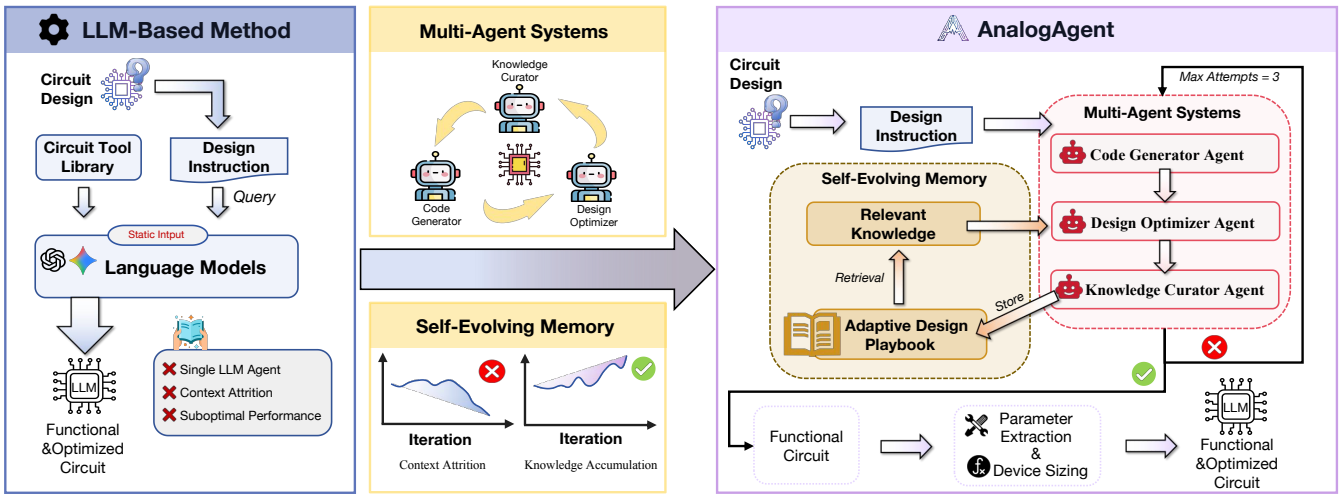


Figure 2: **AnalogAgent Framework Overview. Left**: Existing LLM-based methods typically rely on a single LLM with static prompts, which often produce suboptimal solutions even when augmented with external circuit libraries. **Right**: AnalogAgent coordinates multiple agents and curates reusable design knowledge. It retrieves prior knowledge from SEM to refine instructions and perform self-improvement, improving convergence toward functionally complete circuits without additional libraries.

3 Methodology

3.1 Problem Formulation

We formulate analog circuit design automation as a training-free code generation problem with simulation-based verification. Formally, the input is a task instance $\mathcal{I} = (x, \tau, \Omega, \Phi_{\text{task}})$, where x is the natural-language instruction, τ is the circuit/task type, Ω specifies hard interface/API constraints enforced by the evaluation criteria (e.g., required node names, subcircuit pins, and simulator settings), and $\Phi_{\text{task}} = \{\varphi_k\}_{k=1}^{K}$ is a set of task-specific functional assertions evaluated on simulation outputs. Given $\mathcal{I}$, our AnaglogAgent outputs an executable design candidate $c \in \mathcal{P}$ (SPICE/Python), where $\mathcal{P}$ denotes the space of candidate SPICE/Python programs that satisfy the required interface and API constraints. Executing c yields $(e, s, z) = \mathcal{E}(c)$, where $e \in \{0, 1\}$ indicates program-level or evaluation-criteria results, $s \in \{0, 1\}$ indicates simulator-level results, and z denotes the corresponding simulated results and waveforms used to verify design requirements. A candidate design is valid only if all hard constraints and task assertions hold:

$$\text{PASS}(c) = 1 \iff (e = 0) \ \wedge \ (s = 0) \ \wedge \ \Big(\bigwedge_{k=1}^{K} \varphi_k(z) = 1 \Big). \quad (1)$$

Violations of Ω are captured by $e = 1$ or $s = 1$. In summary, the objective is to map $\mathcal{I}$ to a candidate $c \in \mathcal{P}$ such that $\text{PASS}(c) = 1$.

3.2 Framework Overview

In this paper, we present AnalogAgent, a training-free agentic framework for analog circuit design automation that integrates a multi-agent system (MAS) with self-evolving memory (SEM). As shown in Figure 2, the MAS consists of three specialized agents for code generation, circuit design optimization, and knowledge curation, respectively. Through iterative self-improvement, the MAS progressively consolidates acquired circuit design knowledge into an Adaptive Design Playbook within the SEM module. This mitigates context attrition via structured, incremental updates that preserve fine-grained circuit design knowledge while ensuring contextual representations remain comprehensive during the learning process. A more detailed workflow is provided in Appendix C. We detail each component in the following subsections.

3.3 Multi-agent Systems for Circuit Design

In analog design tasks, context attrition is particularly acute as the LLM agent must (i) generate executable SPICE/Python code under strict interface constraints across repeated revisions, and (ii) preserve simulation-critical details rather than collapsing them into shorter, less informative summaries that dilute the original instructions and obscure useful guidance. This motivates the design of MAS that decouples generation, optimization, and knowledge curation into a structured workflow with persistent memory. MAS maintains a continuously evolving Adaptive Design Playbook that stores distilled, validated heuristics and design constraints. Consequently, the Code Generator Agent conditions on task-relevant knowledge retrieved from this playbook, instead of repeatedly reconstructing instructions from transient context.

Code Generator Agent. Given a natural-language instruction x, the code generator produces task-specific SPICE/Python design scripts. Let $t \in \{1, 2, \dots\}$ index the iteration. The code generation prompt P_t is constructed by augmenting the task instruction with domain knowledge retrieved from the Adaptive Design Playbook:

$$P_t = P_{\text{task}}(x) \oplus r_t, \qquad r_t = \mathcal{R}(M_t, \tau), \quad (2)$$

where $P_{\text{task}}(x)$ is the task requirements; r_t is the retrieved task-specific knowledge; $\mathcal{R}(\cdot)$ is the retrieval operator; M_t is the SEM state (i.e., the playbook) at iteration t; and τ denotes the circuit/task type. The code generator then produces a candidate script c_t:

$$c_t = \mathcal{G}(P_t), \quad (3)$$

where $\mathcal{G}$ denotes the code generator agent.

Design Optimizer Agent. After generating the script, a design optimizer agent evaluates it and converts execution feedback into actionable guidance for circuit revision. Specifically, it performs multi-stage checks: (i) requirement compliance; (ii) DC feasibility via simulation and operating-point inspection; (iii) DC-sweep transfer validation; (iv) task-specific functional tests; and (v) waveform sanity checks [12, 13]. Formally, given a candidate script c_t, the execution routine returns structured signals:

$$(e_t,\ s_t,\ \ell_t,\ z_t) = \mathcal{E}(c_t), \tag{4}$$

where $\mathcal{E}(\cdot)$ denotes the execution and evaluation routine; $e_t \in \{0, 1\}$ indicates whether a runtime error occurred at iteration t ($e_t = 1$ for failure, $e_t = 0$ otherwise); $s_t \in \{0, 1\}$ denotes whether a simulator-level failure occurred; ℓ_t is the diagnostic log used for feedback; and z_t denotes simulated measurements and waveforms used to verify design requirements. A candidate is accepted if Eq. (1) evaluates to true, yielding $\mathtt{PASS}(c_t)$.

Noted that for circuit classes requiring transfer-curve validation, the design optimizer additionally performs a DC sweep on the current candidate c_t to identify a feasible bias point, and constructs an auxiliary bias-updated candidate for re-evaluation:

$$v_t = \mathcal{B}(\mathcal{D}(c_t)), \qquad \tilde{c}_t = \mathcal{T}(c_t, v_t), \tag{5}$$

where $\mathcal{D}(\cdot)$ produces sweep results for c_t, $\mathcal{B}(\cdot)$ selects a feasible bias point from these results, and $\mathcal{T}(\cdot,\cdot)$ injects the selected bias.

Knowledge Curator Agent. Inspired by recent agentic frameworks that store salient contextual information for accumulate, retrieve, and reason over long-horizon experiences [1, 28, 34], we propose a Knowledge Curator that consolidates the design optimizer's feedback into SEM by maintaining and refining an Adaptive Design Playbook. Rather than rewriting the full context, it performs incremental updates that extract compact, reusable rules and code-level patterns from both failures and successes. This preserves validated guidance while incorporating task-driven insights, mitigating context attrition and enabling reliable retrieval of domain-specific tactics for reference. Formally, given a validated feedback bundle, the curator distills reusable knowledge and exemplars conditioned on the circuit/task type τ, which are then incorporated into SEM through incremental memory updates across iterations.

$$f_t = \big(x, c_t, e_t, s_t, \ell_t, z_t, \mathtt{PASS}(c_t)\big), \Delta_t = \phi(f_t), \tag{6}$$

where f_t aggregates the feedback information and $\phi(\cdot)$ maps this feedback to structured circuit design knowledge Δ_t, which is stored in the memory mechanism described in the following subsection.

3.4 Self-Evolving Memory Mechanism

LLMs benefit from long, detailed circuit-design instructions, yet as instruction complexity and iteration depth grow, LLMs exhibit a tendency to compress contexts into distilled summaries, which accelerates context attrition. This motivates the introduction of self-evolving memory (SEM), maintained as an Adaptive Design Playbook that incrementally refines structured reusable circuit design knowledge. Specifically, SEM constitutes a training-free, dynamically updated repository that facilitates cross-task generalization and enhances convergence on complex circuit design tasks. In doing so, it reduces reliance on repeated full-context rewrites and mitigates information loss across iterative self-improvement, while operating without expert feedback or reliance on predefined circuit libraries or databases. Overall, by continuously updating the Adaptive Design Playbook and retrieving task-relevant design knowledge, MAS and SEM establish a closed-loop self-improvement paradigm for analog circuit design automation.

Adaptive Design Playbook Update. AnalogAgent maintains a SEM repository as an Adaptive Design Playbook that accumulates validated rules and exemplars across iterations through incremental, curator-approved updates. Given the circuit design knowledge extracted by the Knowledge Curator Agent, the playbook is continuously updated as follows:

$$M_{t+1} = \mathcal{U}(M_t,\ \Delta_t) = \mathrm{Dedup}\big(\mathrm{Filter}(M_t \parallel \Delta_t; \Omega)\big), \tag{7}$$

where $\mathcal{U}(\cdot)$ updates the playbook M_t with conflict checking under interface/API constraints Ω, followed by filtering, deduplication process to suppress erroneous experience updates, eliminate redundant entries, and maintain a compact, structured memory.

Design Knowledge Retrieval. When encountering a new circuit design task, relevant knowledge is retrieved from the playbook to guide script generation, thereby improving both consistency and efficiency. Given the current task type τ (and optional recent failure reflection $\mathcal{F}$), a retriever first queries task-specific memory in M_t by type match and resorts to substring-based retrieval when such a match is unavailable, and then forms

$$r(\tau) = \begin{cases} \mathcal{R}[\tau], & \tau \in \mathrm{keys}(r), \\ \mathcal{R}[k^*], & k^* = \min\{k \in \mathrm{keys}(r) : k \prec \mathrm{str}(\tau)\}, \\ \varnothing, & \text{otherwise.} \end{cases} \tag{8}$$

where $k \prec \mathrm{str}(\tau_t)$ denotes a substring match i.e., k is contained within the string representation of τ. This prioritizes exact task-aligned matches, while enabling approximate reuse of related entries when exact matches are unavailable, thereby improving robustness and coverage.

Closed-loop Self-improvement. During the self-improving process, AnalogAgent forms a closed loop in which the updated memory state M_{t+1} is retrieved to construct the retrieved knowledge r_{t+1}, which is then combined with the task instruction to form P_{t+1} for generating the next candidate c_{t+1}:

$$\begin{aligned} r_{t+1} &= \mathcal{R}(M_{t+1}, \tau), \\ P_{t+1} &= P_{\mathrm{task}}(x) \oplus r_{t+1}, \\ c_{t+1} &= \mathcal{G}(P_{t+1}). \end{aligned} \tag{9}$$

This completes the loop between generation, verification, and knowledge updates and retrieval. In summary, MAS provides the operational mechanism for producing and applying structured feedback for circuit design knowledge curation, while SEM serves as persistent memory that preserves these updates and supports self-improving across iterations and cross-task knowledge transfer.

3.5 Circuit Optimization

After a feasible circuit design is generated, the design is further refined through parameter optimization. Following AnalogCoder-Pro [13], the circuit is first validated via simulation to ensure executability. Based on the task instruction x and simulated results z_t, the LLM identifies key tunable variables (e.g., device dimensions or component values) together with their corresponding search ranges. Bayesian optimization using the Tree-structured Parzen Estimator

(TPE) implemented in Optuna [9] is then employed to iteratively evaluate candidate parameter configurations through simulation and select the configuration that satisfies the target specifications.

4 Experiments

4.1 Experimental Settings

Benchmark. We evaluate on a 30-task analog circuit benchmark grouped by component count and topological complexity (Tasks 1–8 *Easy*, Tasks 9–13 *Medium*, Tasks 14–30 *Hard*), which is publicly available [1] and adopted from prior work [12, 13]. We provide clear, detailed descriptions of how evaluation data are collected through SPICE/Python execution (e.g., simulation logs, measurements, and waveforms), how these artifacts are preprocessed and managed across runs, and how results are analyzed under a unified protocol to ensure rigorous, reproducible assessment. Please refer to Appendix B for more details.

Baselines. We compare AnalogAgent with **SPICEPilot** [30], **AnalogCoder** [12], and **AnalogCoder-Pro** [13]. SPICEPilot uses prompt-based generation with curated instructions and in-context examples [30]. AnalogCoder introduces a Circuit Tool Library for reusable components and structured synthesis [12], and AnalogCoder-Pro further adds a multimodal feedback-enhanced design flow [13]. The evaluation includes a diverse set of LLMs, including DeepSeek-V2-Lite, Llama-3-8B, Gemini-2.5-Flash, GPT-5. All methods use identical configurations for fair comparisons, with full experimental configurations provided in Appendix B.

Metrics. We follow prior work [12, 13] and use Pass@k to assess the reliability of generated circuit designs, defined as the probability of obtaining at least one valid solution among k samples: Pass@k = $1 - \frac{\binom{n-c}{k}}{\binom{n}{k}}$, where n is the total number of candidates and c is the number of correct solutions. Throughout, we fix $n = 30$ to obtain stable estimates under stochastic model outputs.

4.2 Experimental Results

Table I reports primary results on 30 analog circuit design tasks under a unified benchmark and evaluation protocol, spanning multiple open-source LLMs and LLM baselines. The open-source DeepSeek-V2-Lite achieves Pass@1/Pass@5 of 6.7/11.4, while Llama3-8B attains 8.6/27.0. SPICEPilot exhibits relatively low Pass@1 (50.1) but much higher Pass@5 (96.5), indicating that it often reaches a valid executable solution only after multiple independent samples rather than in the first attempt. Under GPT-5, AnalogCoder performs poorly on Hard tasks (Pass@1 36.5), while AnalogCoder-Pro improves the average Pass@1/Pass@5 to 88.6/98.2 but remains limited on several high-difficulty cases (e.g., Tasks 24–25). On the other hand, AnalogAgent consistently attains the best overall Pass@1 and Pass@5 among all methods. Specifically, with GPT-5 backbone, AnalogAgent achieves 97.4/100.0 Pass@1/Pass@5 and improves Hard-task Pass@1 over AnalogCoder-Pro by around 12.5%. With Gemini-2.5-Flash, AnalogAgent reaches 92.0/99.9, suggesting a lower first-try success rate but a comparable multi-sample ceiling.

Table II complements these aggregate results with statistics over five random trials, reporting Pass@1 as mean±std for Easy/Medium/Hard groups. AnalogAgent achieves the highest mean Pass@1 with smaller variance across trials, including **87.1 ± 2.9** on Medium and **88.7 ± 0.9** on Hard tasks, indicating more stable first-attempt performance in the regimes where prior methods struggle. Beyond success rates, Table II also summarizes efficiency using the average tokens and runtime required to reach the first success across all 30 tasks. AnalogAgent reduces Time-to-First-Success to 1.3 minutes on average (vs. 2.6 for SPICEPilot and 2.1 for AnalogCoder-Pro) and lowers tokens to first success to $(\sim)32K (vs. (\sim)40K$ for AnalogCoder-Pro). Taken together, these results indicate that AnalogAgent's gains are not driven by increased search effort; instead, by structuring reusable knowledge into an Adaptive Design Playbook and retrieving targeted guidance in subsequent iterations, our framework improves both first-shot reliability and convergence efficiency relative to existing baselines.

4.3 Ablation Studies

To assess the contributions of SEM and MAS, we conduct ablations with Gemini-2.5-Flash under four configurations: (i) the baseline Gemini without agentic famework AnalogAgent; MAS only retains multi-agent system with iteration-level feedback exchange, but disables curator-driven memory updates and playbook retrieval; SEM only enables curator-style rule distillation with playbook-guided generation, but removes multi-agent and operates in a single-agent loop. As summarized in Table III, the removal of both modules results in a clear reduction in the success of analog circuit design automation, with the most pronounced degradation observed in high-complexity tasks. On Hard tasks, Pass@1 decreases from 89.8 in the full framework to 43.7 when both modules are removed, and the overall Avg Pass@1/Pass@5 drops from 92.0/99.9 to 63.8/76.8. The SEM is associated with higher performance on Easy tasks by adding revelant knowledge. Easy-task Pass@1 improves from 78.3 to 88.8 with SEM only and reaches 100.0 in the full model. In contrast, MAS shows a larger effect on Medium and Hard tasks, where multi-agent design patterns assist in resolving complex topological constraints and bias relationships. Medium-task Pass@1 increases from 60.0 to 75.3 with MAS only, while Hard-task Pass@1 increases from 34.5 to 68.8. The full framework yields the strongest results among all variants, demonstrating that short-term refinement and long-term information reuse are jointly necessary and most effective when combined.

4.4 Self-Improving Refinement Case Study

To better understand the behavior of the proposed framework during circuit synthesis, this section analyzes representative design trajectories and iterative correction patterns. Task 25, an Op-amp Comparator, is selected as an illustrative example to demonstrate the progression from initial failure to a valid solution. As shown in Figure 3, the process spans 30 iterations, during which successive generations are refined based on execution feedback. Early attempts exhibit library usage errors and netlist construction issues, such as duplicated element definitions. In contrast to baseline frameworks that frequently repeat similar syntax-level mistakes, AnalogAgent shows a progressive reduction of such errors across iterations. Execution feedback highlights concrete problems, which are recorded by the Knowledge Curator as reusable correction rules, including ensuring unique identifiers for MOSFET instances and

[1] https://github.com/laiyao1/AnalogCoder

Table I: **Main results.** Across diverse LLM backbones and model variants under a unified benchmark and evaluation protocol, reporting per-task and total tasks Pass@1 and Pass@5.

Model	DeepSeek-V2-Lite		Llama3-8B		SPICEPilot (Gemini-2.5-Flash)		AnalogCoder (Gemini-2.5-Flash)		AnalogCoder (GPT-5)		AnalogCoder-Pro (Gemini-2.5-Flash)		AnalogCoder-Pro (GPT-5)		AnalogAgent (Gemini-2.5-Flash)		AnalogAgent (GPT-5)	
Task ID	Pass@1	Pass@5	Pass@1	Pass@5	Pass@1	Pass@5	Pass@1	Pass@5	Pass@1	Pass@5	Pass@1	Pass@5	Pass@1	Pass@5	Pass@1	Pass@5	Pass@1	Pass@5
1	83.3	100.0	26.7	81.5	60.0	99.4	73.3	100.0	100.0	100.0	100.0	100.0	100.0	100.0	100.0	100.0	100.0	100.0
2	0.0	0.0	30.0	85.7	63.3	99.7	70.0	99.9	100.0	100.0	100.0	100.0	100.0	100.0	100.0	100.0	100.0	100.0
3	0.0	0.0	20.0	70.2	66.7	99.8	90.0	100.0	100.0	100.0	100.0	100.0	100.0	100.0	100.0	100.0	100.0	100.0
4	0.0	0.0	3.3	16.7	60.0	99.4	73.3	100.0	100.0	100.0	80.0	100.0	100.0	100.0	100.0	100.0	100.0	100.0
5	0.0	0.0	0.0	0.0	63.3	99.7	80.0	100.0	100.0	100.0	93.3	100.0	100.0	100.0	100.0	100.0	100.0	100.0
6	13.3	53.8	26.7	81.5	60.0	99.4	100.0	100.0	100.0	100.0	100.0	100.0	100.0	100.0	100.0	100.0	100.0	100.0
7	0.0	0.0	30.0	85.7	43.3	95.7	100.0	100.0	100.0	100.0	100.0	100.0	100.0	100.0	100.0	100.0	100.0	100.0
8	3.3	16.7	13.3	53.8	40.0	94.0	80.0	100.0	76.7	100.0	76.7	100.0	100.0	100.0	100.0	100.0	100.0	100.0
9	20.0	70.2	0.0	0.0	50.0	97.9	10.0	43.3	96.7	100.0	26.7	81.5	100.0	100.0	66.7	99.8	100.0	100.0
10	0.0	0.0	23.3	76.4	56.7	99.1	96.7	100.0	100.0	100.0	100.0	100.0	100.0	100.0	100.0	100.0	100.0	100.0
11	0.0	0.0	0.0	0.0	33.3	89.1	16.7	62.7	100.0	100.0	100.0	100.0	100.0	100.0	100.0	100.0	100.0	100.0
12	0.0	0.0	0.0	0.0	46.7	96.9	23.3	76.4	43.3	95.7	30.0	85.7	66.7	99.8	70.0	99.9	100.0	100.0
13	0.0	0.0	26.7	81.5	53.3	98.6	33.3	89.1	86.7	100.0	96.7	100.0	100.0	100.0	96.7	100.0	100.0	100.0
14	80.0	100.0	33.3	89.1	63.3	99.7	83.3	100.0	93.3	100.0	100.0	100.0	100.0	100.0	100.0	100.0	100.0	100.0
15	0.0	0.0	20.0	70.2	46.7	96.9	23.3	76.4	50.0	97.9	90.0	100.0	56.7	99.1	100.0	100.0	100.0	100.0
16	0.0	0.0	0.0	0.0	53.3	98.6	0.0	0.0	23.3	76.4	66.7	99.8	16.7	62.7	70.0	99.9	83.3	100.0
17	0.0	0.0	3.3	16.7	43.3	95.7	0.0	0.0	0.0	0.0	76.7	100.0	100.0	100.0	86.7	100.0	90.0	100.0
18	0.0	0.0	0.0	0.0	63.3	99.7	93.3	100.0	90.0	100.0	100.0	100.0	100.0	100.0	100.0	100.0	100.0	100.0
19	0.0	0.0	0.0	0.0	56.7	99.1	0.0	0.0	0.0	0.0	100.0	100.0	100.0	100.0	100.0	100.0	100.0	100.0
20	0.0	0.0	0.0	0.0	53.3	98.6	0.0	0.0	0.0	0.0	100.0	100.0	80.0	100.0	100.0	100.0	83.3	100.0
21	0.0	0.0	0.0	0.0	36.7	91.8	0.0	0.0	0.0	0.0	93.3	100.0	96.7	100.0	100.0	100.0	100.0	100.0
22	0.0	0.0	0.0	0.0	33.3	89.1	0.0	0.0	0.0	0.0	100.0	100.0	90.0	100.0	100.0	100.0	86.7	100.0
23	0.0	0.0	0.0	0.0	50.0	97.9	90.0	100.0	0.0	0.0	83.3	100.0	80.0	100.0	73.3	100.0	100.0	100.0
24	0.0	0.0	0.0	0.0	43.3	95.7	16.7	62.7	0.0	0.0	33.3	89.1	36.7	91.8	86.7	100.0	100.0	100.0
25	0.0	0.0	0.0	0.0	30.0	85.7	26.7	81.5	0.0	0.0	3.3	16.7	40.0	94.0	53.3	98.6	86.7	100.0
26	0.0	0.0	0.0	0.0	33.3	89.1	83.3	100.0	100.0	100.0	96.7	100.0	100.0	100.0	100.0	100.0	100.0	100.0
27	0.0	0.0	0.0	0.0	50.0	97.9	90.0	100.0	100.0	100.0	100.0	100.0	100.0	100.0	100.0	100.0	93.3	100.0
28	0.0	0.0	0.0	0.0	36.7	91.8	0.0	0.0	33.3	89.1	100.0	100.0	96.7	100.0	100.0	100.0	100.0	100.0
29	0.0	0.0	0.0	0.0	56.7	99.1	73.3	100.0	100.0	100.0	100.0	100.0	100.0	100.0	96.7	100.0	100.0	100.0
30	0.0	0.0	0.0	0.0	56.7	99.1	0.0	0.0	30.0	85.7	3.3	16.7	96.7	100.0	60.0	99.8	100.0	100.0
Avg	6.7	11.4	8.6	27.0	50.1	96.5	47.6	66.4	60.8	71.5	81.7	93.0	88.6	98.2	92.0	99.9	**97.4**	**100.0**
Imp	1362.4	780.5	1039.2	270.8	94.5	3.7	104.9	50.6	60.3	39.9	19.3	7.5	10.0	1.8	5.9	0.1	0	0

Table II: Average Pass@1 success rates by task difficulty and average token (K) /runtime (min) cost to first success (mean±std over five trials). All methods use Gemini-2.5-Flash for fairness; NA denotes at least one task unsolved, rendering the aggregate undefined.

Model	Easy	Medium	Hard	Avg Results	Avg Tokens	Avg Time
DeepSeek-V2-Lite	12.7 ± 2.91	10.0 ± 2.9	3.4 ± 1.3	6.4 ± 2.5	NA	NA
Llama3-8B	18.9 ± 11.9	10.1 ± 5.6	1.8 ± 1.2	7.8 ± 4.5	NA	NA
SPICEPilot	48.4 ± 15.0	47.1 ± 13.1	40.7 ± 6.3	43.7 ± 9.3	251.5	2.6
AnalogCoder	87.6 ± 4.8	34.0 ± 4.2	39.1 ± 1.9	50.9 ± 2.6	NA	NA
AnalogCoder-Pro	91.9 ± 3.4	83.2 ± 4.4	73.3 ± 1.7	81.7 ± 1.1	40.2	2.1
AnalogAgent (Ours)	**98.3 ± 1.6**	**87.1 ± 2.9**	**88.7 ± 0.9**	**92.3 ± 0.6**	**32.0**	**1.3**

passing subcircuit objects rather than string labels. In later iterations, the generated topology and pin connections conform to the comparator specification, with V_{out} switching behavior aligned with the condition $V_{in} = V_{ref}$. Notably, AnalogAgent attains a final Pass@1 of 86.7 on Task 25, representing a substantial accuracy improvement over other frameworks on this high-difficulty case.

Throughout the iteration, AnalogAgent first learns the exact number and ordering of positional arguments needed to instantiate each subcircuit topology. When generating the corresponding netlist, it enforces globally unique instance identifiers within each scope to avoid name collisions and to ensure a one-to-one mapping between devices and their stamped mathematical objects during SPICE equation assembly. Building on these rules, AnalogAgent then generates a hierarchical comparator: it defines the "Opamp" subcircuit with a consistent pin order, and then instantiates it at the top level with correctly named, fully connected pins, including supplies and references, yielding a SPICE-simulatable circuit.

4.5 Case Study: Cross-Task Knowledge Transfer

The cumulative experience further supports convergence on related tasks through cross-task reuse of previously recorded design information. For example, information derived from Basic Task 1 regarding explicit supply definition and bulk-tie conventions is retrieved by the Adaptive Design Playbook when addressing Medium Tasks 9 and 10. As illustrated in Figure 4, such reuse preserves consistent biasing strategies and interface naming conventions across tasks, thereby narrowing the range of candidate designs explored for more complex circuits. By incorporating feedback from unsuccessful attempts into updated Playbook entries, the framework replaces isolated sampling with a process guided by stored diagnostic information.

During the generation for Task 1, AnalogAgent learns and retains the circuit-construction grammar required for SPICE netlisting. Concretely, it internalizes the physical pin ordering and connectivity rules for each primitive; for instance, a MOSFET instance is specified in the order of unique identifier, drain, gate, source, body, model name, length, and width, while passive elements require valid two-terminal node connections. It also learns the need for unique instance names (e.g., M1, R1, Vdd) to avoid identifier collisions during parsing and device stamping in simulation. With this

Table III: **Ablation study.** A series of ablation studies on the AnalogAgent built on Gemini-2.5-Flash validate the effectiveness of the proposed method by systematically removing individual components of the framework.

Model	Gemini-2.5-Flash		AnalogAgent (w/o MAS, w/o SEM)		AnalogAgent (w/ MAS, w/o SEM)		AnalogAgent (w/o MAS, w/ SEM)		AnalogAgent	
Task Level	Pass@1	Pass@5	Pass@1	Pass@5	Pass@1	Pass@5	Pass@1	Pass@5	Pass@1	Pass@5
Easy	78.3	85.2	82.9	97.7	94.2	100.0	88.8	98.6	100.0	100.0
Medium	60.0	60.0	64.7	80.0	75.3	92.5	56.0	66.2	86.7	99.6
Hard	34.5	41.0	43.7	52.8	68.8	70.9	45.1	54.3	89.8	100.0
Avg	50.4	56.0	63.8	76.8	79.4	87.8	58.6	68.1	**92.0**	**99.9**

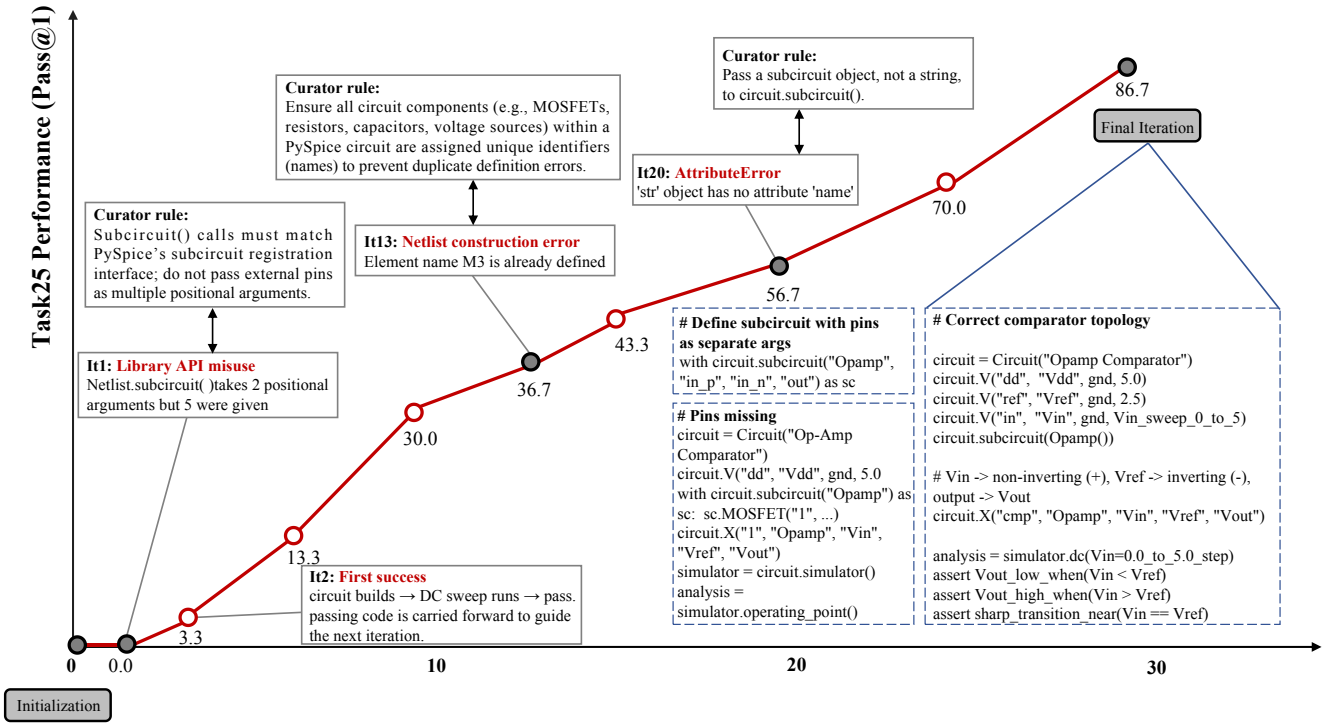


Figure 3: **Self-Improving Refinement Case Study.** The flow shows an error-driven refinement loop for Hard Task 25, in which execution failures are diagnosed, distilled into curator rules, and written into Adaptive Design Playbook to guide subsequent generations and accelerate convergence. Hard Task 25 is an op-amp comparator that outputs high when $V_{\text{in}} > V_{\text{ref}}$ and low when $V_{\text{in}} < V_{\text{ref}}$. The final iterations correct the topology and pin semantics to meet this specification.

foundational "syntax + wiring" knowledge established in Task 1, which implements a simple NMOS pull-down with a resistive pull-up and a safety resistor, the agent transfers the same rules to more complex topologies in Task 9 and Task 10. For Task 9, it reuses the MOSFET netlisting template and expands from a single-stage inverter-like behavior to a two-stage biased CMOS structure by introducing intermediate nodes (Vint1), bias sources (Vbias1, Vbias2), complementary PMOS loads, and a coupling capacitor (Cc), while preserving correct pin order, valid node naming, and model references. For Task 10, it again applies the same instantiation rule-set to generate a regenerative/inverter-style topology (NMOS pull-down plus diode-connected or feedback PMOS load), ensuring the feedback connection, where M2 gate is tied to Vout, remains physically meaningful and simulatable. Overall, AnalogAgent's workflow reflects a progressive transfer from syntax correctness to connectivity consistency and then to topology composition: it first learns how to write valid device lines and assign instance names in Task 1, and subsequently reuses this knowledge to reduce trial-and-error iterations when assembling larger hierarchical or multi-device circuits in Tasks 9 and 10, thereby focusing the remaining search on architectural choices such as biasing, intermediate nodes, and feedback rather than re-learning netlist formatting from scratch.

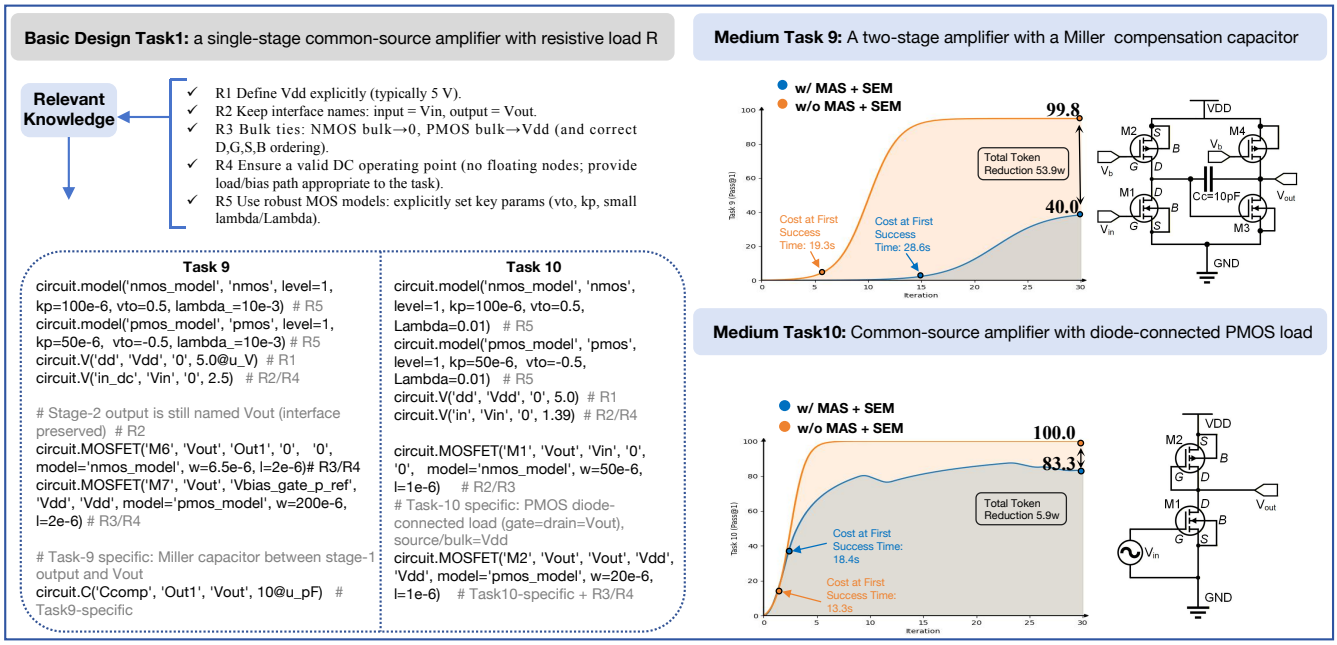


Figure 4: **Cross-Task Transfer across Amplifiers.** Reusable knowledge distilled from the basic Amplifier Task 1 transfers to Tasks 9 and Task 10, improving success rates and demonstrating that long-term memory is effectively reusable across tasks.

4.6 Agentic Reasoning with "Small" LLMs

Commercial IC design often cannot use APIs to access cloud-hosted LLMs because PDKs are proprietary and protected under non-disclosure agreements (NDAs). Therefore, it is important to maintain agent efficiency even when using smaller models, which are more suitable for on-premises or locally hosted servers. To this end, we replace the backbone LLMs with compact open-weight models (i.e., `Qwen3-1.7B`, `Qwen3-4B-Instruct`, `Qwen3-8B`, and `Qwen3-14B`) while keeping the agentic workflows unchanged. As shown in Fig. 5, AnalogAgent achieves higher success rates and solves more tasks than both AnalogCoder-Pro and the corresponding base models on all LLM scales. At the 1.7B scale, Pass@1 improves from 2.4 to 22.2. At 4B and 8B, Pass@1 improves from 28.2→62.3 and 23.3→72.1, respectively. At 14B, Pass@1 further improves from 35.3→76.7. In contrast, AnalogCoder-Pro does not consistently improve performance on compact LLMs and can even degrade performance in certain cases (e.g., Qwen3-14B). Overall, improvements remain consistent across model scales and become more pronounced for larger backbones, indicating that execution-driven feedback and Self-Evolving Memory amplify, rather than substitute for, backbone model capacity. Full results are reported in Appendix G. These findings lend empirical support to the potential of small agents as an efficiency-oriented strategy for practical deployments [2] and further underscore that the benefits of AnalogAgent are not merely a consequence of scaling the backbone model, but arise from the agentic reasoning of iterative feedback and the accumulation and retrieval of reusable design knowledge. From a practical standpoint, this suggests a cost-effective path for industry adoption, as many organizations prefer to deploy compact open-weight models on local servers for privacy, controllability, and reduced inference cost.

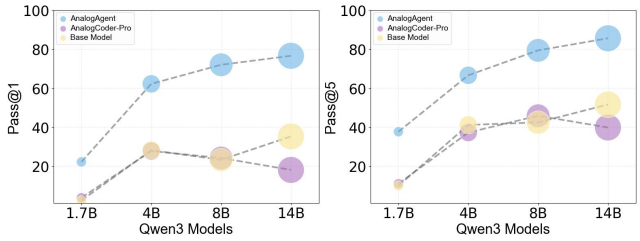


Figure 5: Performance comparison of Qwen base models, AnalogCoder-Pro, and AnalogAgent. AnalogAgent consistently enhances compact Qwen models across all model scales.

5 Conclusion and Future Work

In this work, we propose a training-free agentic framework for analog circuit design automation that seamlessly integrates an LLM-based multi-agent system with a self-evolving memory mechanism. By learning an Adaptive Design Playbook that accumulates reusable design knowledge, AnalogAgent effectively mitigates context attrition and surpasses existing baselines without relying on pre-specified expert knowledge, curated circuit libraries, or external databases. Notably, AnalogAgent substantially improves performance for small open-weight LLMs, spanning 1.7B to 14B Qwen models, indicating that high-quality design reasoning can be achieved even with compact backbones. Looking ahead, this framework opens several avenues for advancing LLM-driven analog design. Future work will expand the evaluation to more complex analog circuits and broader real-world constraints, and will establish new benchmarks that better reflect practical design scenarios.

References

[1] Lakshya A Agrawal, Shangyin Tan, Dilara Soylu, Noah Ziems, Rishi Khare, Krista Opsahl-Ong, Arnav Singhvi, Herumb Shandilya, Michael J Ryan, Meng Jiang, et al. 2025. Gepa: Reflective prompt evolution can outperform reinforcement learning. *arXiv preprint arXiv:2507.19457* (2025).

[2] Peter Belcak, Greg Heinrich, Shizhe Diao, Yonggan Fu, Xin Dong, Saurav Muralidharan, Yingyan Celine Lin, and Pavlo Molchanov. 2025. Small Language Models are the Future of Agentic AI. *arXiv preprint arXiv:2506.02153* (2025).

[3] Ahmet F Budak, Prateek Bhansali, Bo Liu, Nan Sun, David Z Pan, and Chandramouli V Kashyap. 2021. Dnn-opt: An rl inspired optimization for analog circuit sizing using deep neural networks. In *2021 58th ACM/IEEE Design Automation Conference (DAC)*. IEEE, 1219–1224.

[4] Ahmet F Budak, David Smart, Brian Swahn, and David Z Pan. 2023. APOSTLE: Asynchronously parallel optimization for sizing analog transistors using DNN learning. In *Proceedings of the 28th Asia and South Pacific Design Automation Conference*. 70–75.

[5] Chen-Chia Chang, Yikang Shen, Shaoze Fan, Jing Li, Shun Zhang, Ningyuan Cao, Yiran Chen, and Xin Zhang. 2024. Lamagic: Language-model-based topology generation for analog integrated circuits. *arXiv preprint arXiv:2407.18269* (2024).

[6] Zihao Chen, Jiangli Huang, Yiting Liu, Fan Yang, Li Shang, Dian Zhou, and Xuan Zeng. 2024. Artisan: Automated operational amplifier design via domain-specific large language model. In *Proceedings of the 61st ACM/IEEE Design Automation Conference*. 1–6.

[7] Minjeong Choi, Youngchang Choi, Kyongsu Lee, and Seokhyeong Kang. 2023. Reinforcement Learning-based Analog Circuit Optimizer using g m/i D for Sizing. In *2023 60th ACM/IEEE Design Automation Conference (DAC)*. IEEE, 1–6.

[8] Zehao Dong, Weidong Cao, Muhan Zhang, Dacheng Tao, Yixin Chen, and Xuan Zhang. 2023. CktGNN: Circuit graph neural network for electronic design automation. *arXiv preprint arXiv:2308.16406* (2023).

[9] Peter I Frazier. 2018. A tutorial on Bayesian optimization. *arXiv preprint arXiv:1807.02811* (2018).

[10] Jian Gao, Weidong Cao, Junyi Yang, and Xuan Zhang. 2025. AnalogGenie: A generative engine for automatic discovery of analog circuit topologies. *arXiv preprint arXiv:2503.00205* (2025).

[11] Paul R Gray, Paul J Hurst, Stephen H Lewis, and Robert G Meyer. 2024. *Analysis and design of analog integrated circuits*. John Wiley & Sons.

[12] Yao Lai, Sungyoung Lee, Guojin Chen, Souradip Poddar, Mengkang Hu, David Z Pan, and Ping Luo. 2025. Analogcoder: Analog circuit design via training-free code generation. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 39. 379–387.

[13] Yao Lai, Souradip Poddar, Sungyoung Lee, Guojin Chen, Mengkang Hu, Bei Yu, Ping Luo, and David Z Pan. 2025. Analogcoder-pro: Unifying analog circuit generation and optimization via multi-modal llms. *arXiv preprint arXiv:2508.02518* (2025).

[14] Yaguang Li, Yishuang Lin, Meghna Madhusudan, Arvind Sharma, Sachin Sapatnekar, Ramesh Harjani, and Jiang Hu. 2021. A circuit attention network-based actor-critic learning approach to robust analog transistor sizing. In *2021 ACM/IEEE 3rd Workshop on Machine Learning for CAD (MLCAD)*. IEEE, 1–6.

[15] Bo Liu, Yan Wang, Zhiping Yu, Leibo Liu, Miao Li, Zheng Wang, Jing Lu, and Francisco V Fernández. 2009. Analog circuit optimization system based on hybrid evolutionary algorithms. *Integration* 42, 2 (2009), 137–148.

[16] Bingyang Liu, Haoyi Zhang, Xiaohan Gao, Zichen Kong, Xiyuan Tang, Yibo Lin, Runsheng Wang, and Ru Huang. 2025. Layoutcopilot: An llm-powered multi-agent collaborative framework for interactive analog layout design. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* (2025).

[17] Chengjie Liu, Yijiang Liu, Yuan Du, and Li Du. 2024. Ladac: Large language model-driven auto-designer for analog circuits. *Authorea Preprints* (2024).

[18] Jialin Lu, Yijie Li, Fan Yang, Li Shang, and Xuan Zeng. 2023. High-level topology synthesis method for Δ-Σ modulators via bi-level Bayesian optimization. *IEEE Transactions on Circuits and Systems II: Express Briefs* 70, 12 (2023), 4389–4393.

[19] Prabir C Maulik, L Richard Carley, and Rob A Rutenbar. 2002. Integer programming based topology selection of cell-level analog circuits. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 14, 4 (2002), 401–412.

[20] Trent McConaghy, Pieter Palmers, Georges Gielen, and Michiel Steyaert. 2007. Simultaneous multi-topology multi-objective sizing across thousands of analog circuit topologies. In *Proceedings of the 44th annual Design Automation Conference*. 944–947.

[21] Trent McConaghy, Pieter Palmers, Georges Gielen, and Michiel Steyaert. 2008. Automated extraction of expert knowledge in analog topology selection and sizing. In *2008 IEEE/ACM International Conference on Computer-Aided Design*. IEEE, 392–395.

[22] Youngmin Oh, Doyun Kim, Yoon Hyeok Lee, and Bosun Hwang. 2024. CRONuS: Circuit rapid optimization with neural simulator. In *2024 Design, Automation & Test in Europe Conference & Exhibition (DATE)*. IEEE, 1–6.

[23] Pieter Palmers, Trent McConnaghy, Michiel Steyaert, and Georges Gielen. 2009. Massively multi-topology sizing of analog integrated circuits. In *2009 Design, Automation & Test in Europe Conference & Exhibition*. IEEE, 706–711.

[24] Souradip Poddar, Ahmet Budak, Linran Zhao, Chen-Hao Hsu, Supriyo Maji, Keren Zhu, Yaoyao Jia, and David Z Pan. 2024. A data-driven analog circuit synthesizer with automatic topology selection and sizing. In *2024 Design, Automation & Test in Europe Conference & Exhibition (DATE)*. IEEE, 1–6.

[25] Souradip Poddar, Youngmin Oh, Yao Lai, Hanqing Zhu, Bosun Hwang, and David Z Pan. 2025. INSIGHT: A Universal Neural Simulator Framework for Analog Circuits with Autoregressive Transformers. In *2025 62nd ACM/IEEE Design Automation Conference (DAC)*. IEEE, 1–7.

[26] Keertana Settaluri, Ameer Haj-Ali, Qijing Huang, Kourosh Hakhamaneshi, and Borivoje Nikolic. 2020. Autockt: Deep reinforcement learning of analog circuit designs. *arXiv preprint arXiv:2001.01808* (2020).

[27] Yichen Shi, Zhuofu Tao, Yuhao Gao, Tianjia Zhou, Cheng Chang, Yaxin Wang, Bingyu Chen, Genhao Zhang, Alvin Liu, Zhiping Yu, et al. 2025. AMSnet-KG: A netlist dataset for LLM-based AMS circuit auto-design using knowledge graph RAG. *ACM Transactions on Design Automation of Electronic Systems* 30, 6 (2025), 1–37.

[28] Mirac Suzgun, Mert Yuksekgonul, Federico Bianchi, Dan Jurafsky, and James Zou. 2025. Dynamic cheatsheet: Test-time learning with adaptive memory. *arXiv preprint arXiv:2504.07952* (2025).

[29] Petar Veselinovic, Domine Leenaerts, W Van Bokhoven, Francky Leyn, Frank Proesmans, Georges Gielen, and Willy Sansen. 1995. A flexible topology selection program as part of an analog synthesis system. In *Proceedings the European Design and Test Conference. ED&TC 1995*. IEEE, 119–123.

[30] Deepak Vungarala, Sakila Alam, Arnob Ghosh, and Shaahin Angizi. 2024. Spicepilot: Navigating spice code generation and simulation with ai guidance. In *2024 IEEE International Conference on Rebooting Computing (ICRC)*. IEEE, 1–6.

[31] Hanrui Wang, Kuan Wang, Jiacheng Yang, Linxiao Shen, Nan Sun, Hae-Seung Lee, and Song Han. 2020. GCN-RL circuit designer: Transferable transistor sizing with graph neural networks and reinforcement learning. In *2020 57th ACM/IEEE Design Automation Conference (DAC)*. IEEE, 1–6.

[32] Yuxuan Yin, Yu Wang, Boxun Xu, and Peng Li. 2024. Ado-llm: Analog design bayesian optimization with in-context learning of large language models. In *Proceedings of the 43rd IEEE/ACM International Conference on Computer-Aided Design*. 1–9.

[33] Haoyi Zhang, Shizhao Sun, Yibo Lin, Runsheng Wang, and Jiang Bian. 2025. Analogxpert: Automating analog topology synthesis by incorporating circuit design expertise into large language models. In *2025 International Symposium of Electronics Design Automation (ISEDA)*. IEEE, 772–777.

[34] Qizheng Zhang, Changran Hu, Shubhangi Upasani, Boyuan Ma, Fenglu Hong, Vamsidhar Kamanuru, Jay Rainton, Chen Wu, Mengmeng Ji, Hanchen Li, et al. 2025. Agentic context engineering: Evolving contexts for self-improving language models. *arXiv preprint arXiv:2510.04618* (2025).

[35] Zhenxin Zhao and Lihong Zhang. 2020. An automated topology synthesis framework for analog integrated circuits. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 39, 12 (2020), 4325–4337.

A Definitions and Notations

Table IV: Definitions and Notations.

Notations are organized following the execution pipeline of AnalogAgent.

Notation	Definition
S	Circuit specification (task description and pass criteria).
g_i	Sub-goal at iteration i.
K	Max attempts per task (iteration budget).
N	Number of benchmark tasks.
M	Long-term memory of reusable rules and fixes.
E_i	Retrieved memory entries at iteration i.
Δ_i	Prompt directives distilled from E_i.
$\mathcal{P}_i$	Adaptive Design Playbook assembled at iteration i.
F_i	Generated executable design script at iteration i.
L_i	Execution feedback from running F_i.
O_i	Aggregated execution evidence up to iteration i.
$V(\cdot)$	Validator for constraint and functional checks.
v_i	Validation result at iteration i.
m_i	Failure message if v_i is false.
$C(\cdot)$	Constraint checker for structural/API/naming constraints.
c_i	Checker outcome at iteration i (pass/fail and/or violations).
$P(\cdot)$	Simulation procedure (e.g., operating point, DC sweep, transient).
W_i	Waveform evidence at iteration i (if applicable).
δ_i	Targeted fix suggested by the Design Optimizer.
r_i	Curated rule written back to memory.
f_{success}	Whether the task is solved.
T_{TTFS}	Time-to-first-success (elapsed time until first verified success).
CSR(k)	Cumulative success rate within the first k attempts.

B DATA AND REPRODUCIBILITY

This appendix first details the construction, collection, preprocessing, and standardization of the domain-specific benchmark dataset, and then describes the reproducible experimental configuration used in this work. Both the implementation code and benchmark data are released via an anonymous artifact repository to support independent verification, trajectory reconstruction, and full experimental reproduction.

B.1 Domain-Specific Data Integration

Dataset Origin and Domain-Specific Motivation. The benchmark used in this work is constructed from domain-specific analog circuit design datasets originally introduced in prior LLM-based analog design frameworks, including AnalogCoder [12] and AnalogCoder-Pro [13]. These prior works established curated benchmark suites for evaluating code-generation-based circuit synthesis under executable simulation constraints.

Analog circuit design represents a highly domain-specific scientific workflow, where usable data must capture circuit topology, device-level behavior, simulation constraints, and verification criteria simultaneously. General-purpose code corpora or synthetic programming datasets cannot provide sufficient supervision signals for executable circuit synthesis. As a result, the benchmark follows a domain-specific data design principle, emphasizing executable validity, simulator-grounded feedback, and physics-consistent verification signals.

Data Collection and Integration. The final benchmark consists of 30 circuit synthesis tasks derived by consolidating and extending task instances from the AnalogCoder and AnalogCoder-Pro benchmark suites. Task definitions were collected from publicly described benchmark configurations and reconstructed into a unified task specification format. The integrated benchmark spans multiple circuit families, including amplifiers, current mirrors, op-amps, oscillators, filters, comparators, and mixed-signal building blocks.

During integration, task definitions were normalized to ensure consistent specification structure, verification criteria, and execution interfaces. When multiple source benchmarks contained semantically overlapping circuit types, representative task variants were selected based on topology diversity and simulation difficulty, ensuring coverage across open-loop, closed-loop, and multi-block circuit categories.

Data Preprocessing and Normalization. All collected task specifications were preprocessed to ensure compatibility with a unified execution and evaluation pipeline. Preprocessing includes:

- Standardizing circuit specification format, including required node naming conventions and interface definitions;
- Normalizing simulation configuration, including DC operating point setup, transient analysis configuration, and sweep parameter definitions;
- Converting verification criteria into executable checker logic, enabling automated pass/fail evaluation;
- Aligning device model references and simulation library dependencies across tasks.

These preprocessing steps ensure that all tasks can be executed under identical runtime orchestration conditions, eliminating evaluation variance introduced by heterogeneous simulation setups.

Data Management and Artifact Organization. All benchmark tasks are managed using a per-task workspace design. Each workspace stores structured artifacts including task specifications, generated design scripts, simulator execution logs, verification outputs, and optional waveform visualizations. This artifact-level management enables full reconstruction of execution trajectories and supports detailed failure diagnosis analysis. Structured logging is used to record iteration-level execution evidence, including simulator convergence behavior, checker violations, and runtime error signatures. This design ensures that both successful and unsuccessful trajectories can be analyzed systematically.

Data Analysis and Evaluation Signals. Benchmark performance is analyzed using execution-grounded metrics computed across all tasks under a fixed iteration budget. Primary evaluation metrics include Pass@k, cumulative success rate, and Time-to-First-Success. In addition, structured execution evidence is used to analyze failure modes, convergence behavior, and repair effectiveness across

Table V: Benchmark Descriptions. Difficulties are distinguished by background colors (easy, medium, and hard).

Id	Type	Circuit Description	Id	Type	Circuit Description
1	Amplifier	Single-stage CS amplifier with resistive load	16	Oscillator	RC phase-shift oscillator
2	Amplifier	Three-stage CS amplifier with resistive loads	17	Oscillator	Wien Bridge oscillator
3	Amplifier	Common-drain (source follower) amplifier	18	Integrator	Op-amp integrator
4	Amplifier	Single-stage common-gate amplifier	19	Differentiator	Op-amp differentiator
5	Amplifier	Single-stage NMOS cascode amplifier	20	Adder	Op-amp adder
6	Inverter	NMOS inverter with resistive load	21	Subtractor	Op-amp subtractor
7	Inverter	CMOS logical inverter	22	Schmitt	Non-inverting Schmitt trigger
8	CurrentMirror	NMOS constant current source	23	VCO	Voltage-controlled oscillator
9	Amplifier	Two-stage amplifier with Miller compensation	24	PLL	Phase-locked loop
10	Amplifier	CS amplifier with diode-connected PMOS load	25	Comparator	Op-amp comparator
11	Opamp	Differential op-amp with PMOS mirror load	26	Filter	Passive low-pass filter
12	CurrentMirror	Cascode current mirror	27	Filter	Passive high-pass filter
13	Opamp	Single-stage differential CS op-amp	28	BandPass	Passive band-pass filter
14	Opamp	Two-stage differential op-amp with active loads	29	BandStop	Passive band-stop filter
15	Opamp	Telescopic cascode op-amp	30	Mixer	Gilbert cell mixer

iterations. For waveform-sensitive circuits such as oscillators and transient-dependent designs, waveform outputs are additionally generated and used for functional verification and diagnostic interpretation. These signals provide domain-specific evaluation evidence beyond static code correctness.

Extension Beyond Prior Benchmarks. The benchmark used in this work is constructed by consolidating domain-specific circuit synthesis tasks from AnalogCoder and AnalogCoder-Pro into a unified specification and execution framework. The primary contribution is not the introduction of new benchmark tasks, but the standardization of execution interfaces, verification pipelines, and evaluation protocols, enabling systematic study of execution-driven iterative circuit synthesis under a multi-agent feedback-repair paradigm. This unified configuration enables consistent evaluation of code generation quality, execution-grounded diagnosis, and iterative repair effectiveness across heterogeneous model backbones and runtime settings.

As shown in Table V, the benchmark consists of 30 domain-specific circuit synthesis tasks spanning multiple circuit families. Difficulty labels are assigned based on circuit topology complexity, feedback structure, and sensitivity to simulation convergence behavior.

B.2 Implementation and Reproducible Experimental Configuration

This section summarizes key implementation mechanisms and runtime configurations required to reproduce the AnalogAgent experimental pipeline. Detailed source code, runtime scripts, and reproduction instructions are provided in the anonymous artifact repository on github:

https://github.com/artifact-repro/analogagent-artifact

Execution Environment and Reproducibility. All experiments were conducted on Ubuntu 20.04.6 LTS with Python 3.10 using a dedicated Conda environment. The machine learning runtime stack includes CUDA-enabled PyTorch, vLLM for high-throughput inference serving, and Transformers-based model integration. Experiments were executed on a dual-GPU workstation equipped with two NVIDIA RTX A6000 GPUs using CUDA 12.4 and NVIDIA driver 550.54.

To ensure reproducibility, all experiments were executed under fixed iteration budgets and standardized task specifications. Random seeds were fixed where applicable. For each task, the system records intermediate artifacts including generated design scripts, execution logs, verification outputs, and waveform plots when required. These artifacts enable full reconstruction of execution trajectories for both successful and unsuccessful runs, supporting detailed failure analysis and independent verification.

Execution-Driven Multi-Agent Runtime. AnalogAgent is implemented as an execution-grounded multi-agent orchestration pipeline operating on executable circuit artifacts. The runtime integrates specification-conditioned code generation, structured execution feedback collection, rule-level retrieval from Self-Evolving Memory (SEM), and curator-controlled rule consolidation. Internal prompts act as structured interfaces for transporting sub-goals, execution observations, and retrieved rule-level constraints rather than as standalone knowledge sources. Circuit-domain constraints and repair heuristics are stored in SEM and injected through the Adaptive Design Playbook during runtime.

During each iteration, the runtime constructs a task-specific instruction context combining circuit specification, current sub-goal derived from execution observations, and retrieved rule-level SEM entries. Execution feedback including simulator logs, checker outputs, operating-point diagnostics, and waveform evidence is normalized into structured observations and consumed by the Design Optimizer and Knowledge Curator for targeted repair and rule consolidation.

Multi-Backbone Support. AnalogAgent is backbone-agnostic and supports both local open-weight inference and API-based inference. Local experiments use vLLM-based serving, while cloud inference uses vendor SDK integrations. Evaluated backbone families include Qwen3 series (local deployment), Gemini API models, and GPT-series API models. All models are evaluated under identical task settings, iteration budgets, and execution-feedback pipelines.

C ANALOGAGENT INFERENCE WORKFLOW

This appendix details the end-to-end inference workflow of AnalogAgent, describing how multi-agent execution feedback, Self-Evolving Memory (SEM), and Adaptive Design Playbook–guided generation interact during iterative circuit synthesis.

C.1 System Overview and Core Components

AnalogAgent is an execution-driven agentic framework for automated analog circuit design. The system couples iterative code execution, multi-agent diagnosis, and long-term experience accumulation to improve correctness, robustness, and convergence efficiency across heterogeneous circuit tasks. At a high level, AnalogAgent operates through three tightly coupled layers:

(1) Self-Evolving Memory (SEM). SEM is a persistent rule-level long-term memory that stores curated and transferable knowledge distilled from historical execution trajectories. Instead of storing raw traces or full prompts, SEM maintains compact structured rules such as validated repair patterns, topology-specific constraints, and stable checker-compliant design practices. SEM corresponds to the long-term memory M used during inference.

(2) Adaptive Design Playbook. The Adaptive Design Playbook is a per-iteration instruction layer constructed dynamically by retrieving relevant entries from SEM and combining them with iteration-specific design constraints. The playbook contains: (i) *Design Instruction*, which encodes task-specific requirements such as node naming, supply conventions, and functional verification constraints; and (ii) *Relevant Knowledge*, which includes transferable experience such as previously validated fixes and task-type-specific heuristics. The playbook therefore acts as a task-conditioned operational view over SEM.

(3) Multi-Agent System (MAS). AnalogAgent adopts a tight, three-agent refinement loop operating over executable artifacts and execution traces at runtime:

- **Code Generator Agent:** Generates executable PySpice circuit implementation and testbench code conditioned on the specification and playbook directives.
- **Design Optimizer Agent:** Diagnoses execution failures using structured signals such as checker outputs, simulation diagnostics, and waveform evidence, and proposes concrete, targeted fix directives.
- **Knowledge Curator Agent:** Converts evidence-backed fixes into transferable rule entries through filtering, conflict checking, and deduplication before writing them into SEM.

To control computational cost and standardize evaluation, each task is associated with a fixed iteration budget (Max Attempts), which caps the number of generate–execute–refine cycles.

C.2 Execution-Driven Iterative Inference Loop

The AnalogAgent inference procedure follows an iterative design – execute – diagnose – refine cycle.

Sub-goal formation and experience retrieval. At the beginning of each iteration, the system aggregates execution observations (e.g., checker violations, simulation results, diagnostic messages, and waveform anomalies) together with the original circuit specification to form the next actionable sub-goal. The playbook then retrieves a small set of relevant SEM entries and distills them into concise prompt directives, forming structured delta context injected into the generation prompt.

Executable generation and validation-driven feedback. Conditioned on the specification and playbook directives, the Code Generator produces an executable PySpice design script. The script is executed to produce multi-source feedback signals, including: netlist-level structural checks, operating-point and DC sweep diagnostics, functional verification outcomes, and waveform evidence when applicable. A validation module enforces structural correctness, connectivity integrity, and task-level functional constraints. When validation fails, failure messages are propagated as structured feedback for the next iteration rather than triggering full prompt re-generation.

Failure diagnosis, rule curation, and memory evolution. The Design Optimizer analyzes full execution trajectories (code, logs, checker output, and waveform evidence) to localize root causes and produce minimal targeted fix directives. The Knowledge Curator then converts evidence-backed fixes into admission-controlled rule entries and writes them into SEM. This separation between short-term iterative refinement and long-term rule consolidation prevents prompt bloat and preserves stable task-specific constraints across successive iterations.

Workspace materialization and traceability. For reproducibility and debugging, the system materializes per-iteration workspace artifacts, including task specification, sub-goal history, retrieved memory entries, execution logs, waveform plots, and generated executable scripts.

A formal algorithmic description is provided in Algorithm 1.

C.3 Multi-Agent Execution Process Visualization

Figure 6 provides a system-level visualization of the multi-agent execution loop. The Code Generator produces executable circuit implementations under specification and playbook guidance. The Design Optimizer closes the execution feedback loop by performing requirement validation, simulation diagnostics, and curve-level waveform analysis when applicable, and converts failure evidence into targeted fix directives. The Knowledge Curator aggregates diverse multimodal execution evidence and writes conflict-checked transferable rules into SEM, enabling cross-iteration and task-level knowledge reuse.

D Algorithmic Procedure

Algorithm 1: AnalogAgent Inference Process

Input: Circuit specification S
Output: Final executable design script F_{exec} and verified outputs
Initialize aggregated observations $O \leftarrow \emptyset$;
$f_{\text{success}} \leftarrow$ false;
attempts $\leftarrow 0$;
while $\neg f_{success}$ ***and*** *attempts* $< K$ **do**
  // Memory-guided planning
  Form the next actionable sub-goal g from S and O;
  Retrieve relevant memory entries E (design rules, constraints, and validated repair patterns);
  Distill E into concise directives Δ;
  // Code generation and execution
  Generate executable PySpice script $F_{\text{exec}} \leftarrow \textsc{CodeGenerator}(S, g, \Delta)$;
  Execute F_{exec} to obtain feedback L (checker diagnostics, simulation outcomes, and verification signals);
  Update observations $O \leftarrow O \cup \{L\}$;
  // Validation and diagnosis
  $v, m \leftarrow \textsc{Validate}(S, O)$;
  if $\neg v$ **then**
    $\delta \leftarrow \textsc{DesignOptimizer}(S, F_{\text{exec}}, O, m)$;
    Use δ to guide regeneration in the next iteration;
  // Curation and memory update
  if *new actionable evidence is identified* **then**
    $r \leftarrow \textsc{Agent}(S, O, v)$;
    Write r into long-term memory and update the playbook;
  attempts $\leftarrow$ attempts +1;
  // Stop criterion
  $f_{\text{success}} \leftarrow \textsc{IsSolved}(S, O)$;
return final executable design script and verified outputs

The AnalogAgent inference process is summarized in Algorithm 1. Given a circuit specification, the system performs iterative memory-guided synthesis through a generate–execute–diagnose–refine loop under a fixed attempt budget.

At each iteration, the system constructs an Adaptive Design Playbook by retrieving transferable rule-level knowledge from Self-Evolving Memory and distilling it into actionable generation directives. The Code Generator produces an executable circuit implementation conditioned on the specification and playbook constraints, and execution feedback is collected through structural checks, simulation diagnostics, and functional verification. When validation fails, the Design Optimizer proposes localized repair directives that guide regeneration in subsequent iterations. Evidence-backed repair patterns are optionally consolidated into memory through curator-controlled filtering and deduplication, enabling cross-task knowledge accumulation without rewriting existing rules.

The inference loop terminates when a verified design is obtained or when the maximum attempt budget is reached. Because memory updates are strictly incremental and retrieval is deterministic, the framework maintains stable constraint conditioning across iterations while allowing new evidence-backed knowledge to be incorporated over time.

E CONTEXT ATTRITION CASE STUDY ON TASK 25)

E.1 Checkpoint Evidence of Context Attrition

Context Attrition Case Study (Task 25): Checkpoint Evidence

[Representative Iterations]
it1: **4.1K** prompt tokens
it10: **2.4K** prompt tokens
it30: **1.4K** prompt tokens

[Attrition Pattern]
Type-1 (Executable feedback → removed): simulator-level convergence traces are progressively pruned.
Type-2 (Concrete diagnosis → abstract rule): failure mode evidence is replaced by generic MOSFET operating principles.
Type-3 (Circuit semantics → surface exception): the retained context collapses to a Python runtime error without analog debug cues.

[Evidence @it1: Simulator-grounded diagnostics]
– Failure mode indicator: "Warning: singular matrix: check nodes vin and vin"
– Solver trajectory: gmin stepping → dynamic gmin stepping → source stepping
– Termination signals: "doAnalyses: iteration limit reached"; "run simulation(s) aborted"

[Evidence @it10: Abstracted operating-point rules]
– Rule-of-thumb constraints: "ensure $V_{GS} > V_{TH}$"; "ensure $V_{DS} > V_{GS} - V_{TH}$"
– Loss: convergence trace and node-level singularity context no longer retained as an executable debugging trajectory

[Evidence @it30: Surface-level exception]
– Runtime symptom only: "KeyError: 'Source_M1_M2'"
– Loss: no simulator-level evidence (e.g., singular matrix / stepping / abort reason) to diagnose the original circuit failure

[Conclusion: Context Attrition]
Across iterations, the prompt compresses from it1 to it30 while shifting away from simulator-grounded evidence. As a result, key task information and diagnostic cues are no longer retained in full, leaving an underspecified context that weakens targeted debugging and subsequent corrections.

E.2 Mechanism and Impact of Context Attrition

In Task 25, context attrition manifests as a shift from simulator-grounded executable feedback (convergence traces) to abstract rule-of-thumb diagnostics and finally to a superficial runtime exception, progressively erasing the domain-specific evidence.

We present a checkpoint-based case study to illustrate context attrition under iterative LLM prompting. On Task 25, the retained context exhibits a pronounced brevity bias, with the prompt tokens shrinking from 4.1k at it1 to 1.4k at it30, a 65.9% reduction relative to it1. This compression is not merely stylistic, as it progressively removes executable feedback that encodes domain-specific debugging value. At it1, the prompt preserves simulator-level evidence,

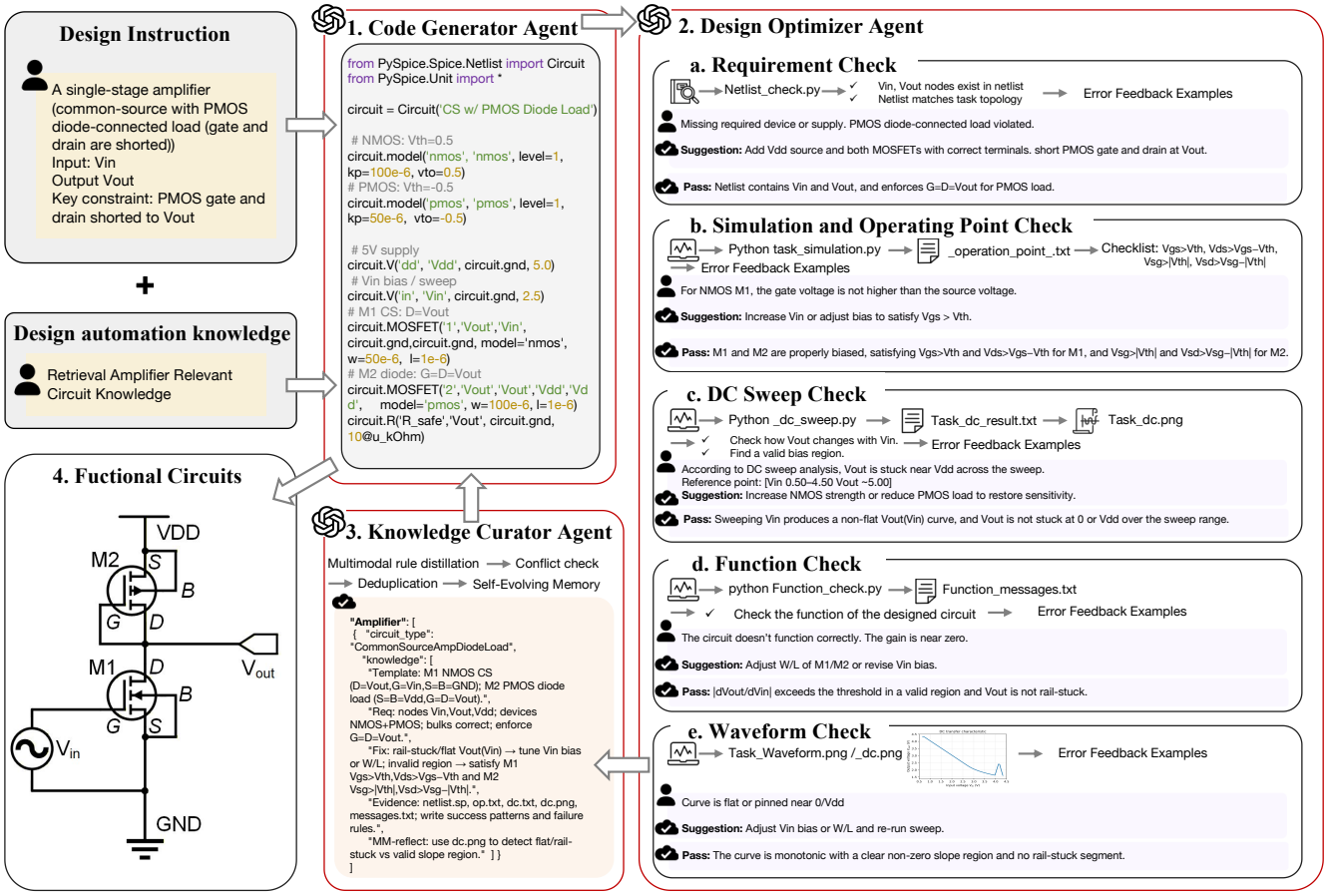


Figure 6: **AnalogAgent workflow with MAS–SEM coupling.** The system solves each task by iterating an inner *Multi-Agent System (MAS)* loop (Code Generator → Design Optimizer → Knowledge Curator) over executable PySpice artifacts. Execution feedback (checker logs, operating-point/DC-sweep diagnostics, and waveform evidence when available) is used for diagnosis and targeted fixes. The curator distills evidence-backed rules with conflict checking and deduplication, writes transferable entries into *Self-Evolving Memory (SEM)*, and the retrieved rules are injected as playbook-style constraints to guide subsequent iterations and related tasks.

including ngspice convergence traces such as gmin stepping, explicit failure indicators such as singular matrix and check nodes, escalation events such as dynamic gmin stepping failed and starting source stepping, and the termination rationale such as iteration limit reached and simulation aborted. By it10, the prompt is shorter and increasingly abstracts these trajectories into higher-level guidance, retaining task keywords such as Vin, Vref, Vout, sweep, and comparator while omitting parts of the diagnostic chain. By it30, the long-form convergence trace is no longer retained, leaving mainly high-level task framing without the fine-grained simulator feedback required for targeted fixes. These checkpoints concretely demonstrate context attrition, where repeated rewrites compress and smooth specialized diagnostic details, thereby weakening the prompt as a carrier of domain expertise across iterations. The Case 25 trajectory also demonstrates how failure evidence can be distilled into reusable diagnostic constraints through SEM-mediated consolidation, grounded in concrete runtime observations rather than hypothetical cross-task transfer scenarios.

F SELF-EVOLVING MEMORY AND ADAPTIVE DESIGN PLAYBOOK

F.1 Memory Entry Semantics

Self-Evolving Memory (SEM) stores *transferable diagnostic rules* distilled from execution trajectories, rather than free-form narrative summaries. Each entry is intended to be actionable in subsequent runs: it links an observable *failure signature* (or other salient execution evidence) to a corresponding *corrective heuristic* that can be applied as a localized patch. The primary evidence sources include simulator logs, checker outputs, operating-point and sweep diagnostics, and (when applicable) waveform plots. This evidence-grounded representation encourages precise reuse: later iterations can condition generation on concrete "what to check" and "what to change" guidance under specified observed conditions, instead of re-deriving procedures from scratch.

For consistency and auditability, entries are kept atomic and scoped. Operationally, an entry can be read as a compact mapping: *Trigger* (task type and/or observed signature) → *Evidence* (what was observed) → *Rule/Patch* (what to enforce or modify) → *Applicability* (when the rule should be invoked). Atomic scoping also supports compositional retrieval: a small subset of entries can be assembled into iteration directives without expanding the prompt with broad, loosely relevant text. Importantly, SEM primarily accumulates failure-derived corrective rules rather than full successful circuit designs. Successful designs are used as short-term guidance within the same task but are not directly written into SEM as reusable rules unless they expose transferable structural constraints or stability guarantees. This design choice prevents overfitting memory to task-specific topology details and improves cross-task generalization.

F.2 Memory Format and Example

Curator-Processed Long-Term Memory Structure Illustration

```
[General Rules]
- Outputs must be named exactly: Voutp and Vout.
- NMOS bulk must tie to source; PMOS bulk must tie to
Vdd.
- CRITICAL API: Never use circuit.add_nodes(); nodes are
created implicitly.
- CRITICAL API: Capacitor initial condition must use
ic=... (not initial_condition=...).
- Opamp subcircuit interface: exactly three pins
(non-inv, inv, out); do not redefine Opamp.

[Task-Type Rules: Comparator]
- Subcircuit pin names must be passed as a single
iterable to subcircuit().
- Any circuit.X(...) instantiation requires the
subcircuit to be defined/loaded beforehand.

[Task-Type Rules: Oscillator]
- Place each Python statement on its own line (avoid
concatenation causing SyntaxError).
- Apply initial conditions to active loop nodes
(kickstart), not fixed supply/bias nodes.
- Ensure every node has a DC path to a reference for
operating-point convergence.

[Task-Type Rules: Integrator / Differentiator]
- Strictly follow explicit checker constraints on allowed
op-amp modeling; prefer task-specific requirements over
generic API rules.
- Keep statement separation strict to avoid SyntaxError.
```

The box illustrates the curator-processed memory structure used for retrieval, organized into general rules and task-type rules. General rules encode API constraints and invariant circuit conventions, while task-type rules capture failure signatures and corrective constraints commonly observed for a circuit family. Not all execution experiences are written into SEM. The Knowledge Curator applies admission control before memory insertion. A candidate rule must satisfy at least one of the following criteria: (i) resolves a repeated failure pattern across iterations, (ii) enforces checker or simulator constraints violated in multiple attempts, (iii) represents a stable design practice independent of specific parameter values. This admission process prevents noisy or task-specific artifacts from polluting long-term memory.

To control memory growth, SEM stores compact rule-level entries rather than full trajectories, and lightweight deduplication and conflict filtering are applied during curation. Retrieval is bounded to a small number of high-relevance entries per iteration.

F.3 Success vs. Failure Evidence Handling

The workflow produces two categories of reusable information: (i) *evidence-backed repair rules* derived from failures and verification violations, and (ii) *stabilizing patterns* observed in verified runs that help maintain invariants across subsequent iterations. The update policy is intentionally conservative about what is promoted into SEM: only entries that can be stated as transferable constraints or diagnostic procedures, and that are supported by concrete execution evidence, are admitted. In contrast, task-specific incidental details that do not generalize beyond a single run are treated as short-term context (e.g., reference artifacts within the current iteration loop) rather than long-term rules.

This separation supports two objectives. First, failure-driven rules reduce repeated mistakes by encoding specific signatures and fixes (e.g., "operating-point does not converge due to missing DC path" → "add a bias resistor or reference path"). Second, stabilizing patterns from verified executions preserve conventions that are easy to regress under iterative prompting (e.g., required node naming and API usage invariants). In both cases, the decision to store information is guided by transferability and evidence: the system prefers compact, auditable entries that can be retrieved and assembled into iteration directives without inflating the prompt with full code histories.

F.4 Curator Filtering and Update Policy

The Knowledge Curator enforces a lightweight but explicit admission pipeline before writing to SEM. Candidate entries are first checked for conflicts with invariant conventions (e.g., required node naming and API constraints) and for logical consistency with existing rules. Entries that duplicate recently stored guidance are filtered through semantic de-duplication to avoid accumulating redundant phrasing of the same constraint. Finally, accepted entries are normalized into an itemized representation so that retrieval can remain targeted and compositional.

Updates are incremental: newly admitted entries are appended as additional rules rather than triggering wholesale rewriting of the memory. This conservative update policy is designed to preserve stable, previously validated conventions while still incorporating new, evidence-backed heuristics as they arise during execution. As a result, the playbook built from retrieved entries remains interpretable and reproducible across iterations: the same core constraints persist, and only localized, verifiable additions are introduced over time.

Table VI: Per-task Pass@k performance comparison across small-scale LLM backbones. Results report AnalogCoder, AnalogCoder-Pro, and AnalogAgent performance on 30 benchmark circuit design tasks.

Model (Avg | Solved)	Metric	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30
Qwen3-1.7B	Pass@1	0	0	0	3.3	3.3	3.3	3.3	0	0	13.3	13.3	3.3	3.3	23.3	3.3	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Avg: 2.4/10.0 | Sol: 10/30	Pass@5	0	0	0	16.7	16.7	16.7	16.7	0	0	53.8	53.8	16.7	16.7	76.4	16.7	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
AnalogCoder-Pro (Qwen3-1.7B)	Pass@1	0	0	0	3.3	10	0	0	3.3	0	0	26.7	10	6.7	53.3	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Avg: 3.8/11.0 | Sol: 7/30	Pass@5	0	0	0	16.7	43.3	0	0	16.7	0	0	81.5	43.3	31	98.6	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
AnalogAgent (Qwen3-1.7B)	Pass@1	53.3	0	0	56.7	33.3	80	3.3	33.3	0	90	43.3	83.3	10	76.7	80	0	0	0	0	0	0	0	0	0	0	53.3	0	0	0	0
Avg: 22.2/37.7 | Sol: 13/30	Pass@5	98.6	0	0	99.1	89.1	99.9	16.7	89.1	0	100	95.5	99.9	43.3	99.9	99.9	0	0	0	0	0	0	0	0	0	0	98.6	0	0	0	0
Qwen3-4B	Pass@1	23.3	0	0	56.7	0	66.7	86.7	100	30	100	93.3	66.7	40	70	0	0	0	0	0	0	0	0	0	0	0	0	0	0	86.7	26.7
Avg: 28.2/41.2 | Sol: 13/30	Pass@5	76.4	0	0	99.1	0	99.8	100	100	85.7	100	100	99.8	93.9	99.9	0	0	0	0	0	0	0	0	0	0	0	0	0	0	100	81.5
AnalogCoder-Pro (Qwen3-4B)	Pass@1	70	63.3	0	73.3	3.3	100	83.3	93.3	0	3.3	83.3	86.7	50	100	20	0	0	0	3.3	0	0	0	0	0	0	0	0	0	0	0
Avg: 27.8/37.3 | Sol: 14/30	Pass@5	99.9	99.7	0	100	16.7	100	100	100	0	16.7	100	100	97.9	100	70.2	0	0	0	16.7	0	0	0	0	0	0	0	0	0	0	0
AnalogAgent (Qwen3-4B)	Pass@1	100	100	100	100	100	100	100	100	86.7	100	100	100	100	93.3	83.3	0	0	0	66.7	0	0	0	0	0	0	93.3	70	0	76.7	100
Avg: 62.3/66.7 | Sol: 20/30	Pass@5	100	100	100	100	100	100	100	100	100	100	100	100	100	100	99.9	0	0	0	99.8	0	0	0	0	0	0	100	99.9	0	99.9	100
Qwen3-8B	Pass@1	40	3.3	26.7	60	26.7	43.3	56.7	96.7	3.3	53.3	50	46.7	0	70	63.3	0	0	60	0	0	0	0	0	0	0	0	0	0	0	0
Avg: 23.3/42.5 | Sol: 15/30	Pass@5	93.9	16.7	81.2	99.5	81.5	95.5	99.1	100	16.7	98.6	97.3	96.6	0	99.9	99.8	0	0	99.5	0	0	0	0	0	0	0	0	0	0	0	0
AnalogCoder-Pro (Qwen3-8B)	Pass@1	33.3	43.3	36.7	90	33.3	40	40	96.7	0	6.7	40	56.7	50	66.7	60	0	0	30	3.3	0	0	0	0	0	0	0	0	0	0	0
Avg: 24.2/45.9 | Sol: 16/30	Pass@5	89.1	95.7	91.8	100	89.1	94	94	100	0	31	94	99.1	97.9	99.8	99.4	0	0	85.7	16.7	0	0	0	0	0	0	0	0	0	0	0
AnalogAgent (Qwen3-8B)	Pass@1	100	100	100	100	93.3	100	96.7	100	96.7	100	90	100	100	80	96.7	0	76.7	63.3	100	0	0	50	33.3	0	0	100	93.3	0	93.3	100
Avg: 72.1/79.5 | Sol: 24/30	Pass@5	100	100	100	100	100	100	100	100	100	100	100	100	100	99.9	100	0	99.9	99.8	100	0	0	97.3	89.1	0	0	100	100	0	100	100
Qwen3-14B	Pass@1	90.0	3.3	96.7	83.3	93.3	83.3	96.7	56.7	50.0	76.7	63.3	70.0	63.3	40.0	90.0	0	0	10.0	0	0	0	0	0	0	0	0	0	0	20.0	6.7
Avg: 35.3/51.7 | Sol: 18/30	Pass@5	100	16.7	100	100	100	100	100	99.1	97.9	100	99.7	99.9	99.7	94.0	100	0	0	43.3	0	0	0	0	0	0	0	0	0	0	70.2	31.0
AnalogCoder-Pro (Qwen3-14B)	Pass@1	73.3	53.3	36.7	83.3	30	50	50	10	3.3	3.3	26.7	33.3	23.3	13.3	20	0	0	23.3	3.3	0	6.7	0	0	0	0	0	0	0	0	0
Avg: 18.1/39.9 | Sol: 18/30	Pass@5	100	98.6	91.8	100	85.7	97.9	97.9	43.3	16.7	16.7	81.5	89.1	76.4	53.8	70.2	0	0	76.4	16.7	0	31	0	0	0	0	0	0	0	0	0
AnalogAgent (Qwen3-14B)	Pass@1	100	100	100	100	100	93.3	100	100	96.7	80.0	73.3	100	86.7	80.0	100	66.7	0	100	70.0	66.7	33.3	100	83.3	0	0	100	73.3	0	96.7	100
Avg: 76.7/85.7 | Sol: 26/30	Pass@5	100	100	100	100	100	100	100	100	100	100	100	100	100	100	100	99.8	0	100	99.9	99.8	89.1	100	100	0	0	100	100	0	100	100

G ADDITIONAL RESULTS ON COMPACT LLMS

Table VI reports detailed per-task results under small and mid-scale backbone models. This table is provided to complement the main experimental results and to demonstrate that the performance gains of AnalogAgent are not restricted to large-capacity models, but remain consistent across a wide range of backbone sizes. All models are evaluated under the same iterative execution-driven synthesis pipeline and identical attempt budgets. Pass@k is computed over iterative attempts rather than independent sampling, and therefore reflects the cumulative probability that a task is solved within the first k execution-refinement iterations. For each model, "Avg" denotes the average Pass@1 across all tasks, while "Solved" indicates the number of benchmark tasks for which at least one valid solution is found within the maximum attempt budget. Columns 1–30 correspond to individual benchmark tasks. Rows group results by backbone model and framework variant, including the base model, AnalogCoder-Pro, and AnalogAgent.

Received 20 February 2007; revised 12 March 2009; accepted 5 June 2009

A 10-Bit 500-MS/s Pipelined SAR ADC With Feedback Factor Compensation in 6-nm FinFET

Yigi Kwon, *Graduate Student Member, IEEE*, Jongyoon Won, *Graduate Student Member, IEEE*, Byounghan Min, Dooyeoun Kim, Jihyun Kim, Jeong-Hyu Yang, and Youngcheol Chae, *Senior Member, IEEE*

***Abstract*—This brief introduces an energy-efficient 10-bit 500-MS/s pipelined SAR ADC that uses feedback factor compensation in 6-nm FinFET technology. The design challenges of residue amplifier in FinFET technology are adequately addressed by incorporating feedback factor compensation. This includes a dynamic negative capacitance circuit at the virtual ground that compensates for the feedback factor and relaxes the requirements of the residue amplifier. This enables the use of an inverter-based residue amplifier that can achieve a high-speed operation of 500-MS/s at a low supply voltage of 0.9 V. The prototype ADC is fabricated in a 6-nm FinFET and occupies 0.014 mm^2. With a Nyquist input signal, it achieves an SNR of 54.2 dB and an SNDR of 53.6 dB, while consuming 2.7 mW from a 0.9 V supply voltage. This brief achieves a competitive Walden figure of merit (FoM) of 13.8 fJ/conv.-step.**

***Index Terms*—Analog-to-digital converter (ADC), 6-nm FinFET process, pipelined SAR ADC, feedback factor compensation, negative capacitance (NC), NC-assisted residue amplifier.**

I. Introduction

IN ADVANCED process nodes, e.g., sub-10 nm process, the level of integration can be significantly increased, but this introduces several challenges for ADC design. For instance, it becomes increasingly difficult to achieve high open-loop gain (A_{OL}) due to the reduction in supply voltage. In addition, design constraints arise from the significant increase in parasitic components. As shown in Table I, the advanced nodes of FinFET technology exhibit a significant increase in parasitic resistance R_P due to additional layers in the fabrication process [1]. An alternative is to utilize wider metal interconnects or additional fins; however, this comes at the cost of increased parasitic capacitance C_P. Consequently, designing using sub-10 nm process poses significant challenges.

Received 2 January 2025; revised 25 February 2025; accepted 20 March 2025. Date of publication 24 March 2025; date of current version 2 May 2025. This work was supported in part by LG Electronics, and in part by the Basic Science Research Program through the National Research Foundation of Korea (NRF) funded by the Ministry of Science and Information and Communication Technology under Grant NRF-2023R1A2C3005110. This brief was recommended by Associate Editor P. C. C. de Aguirre. *(Corresponding author: Youngcheol Chae.)*

Yigi Kwon is with the Department of Electrical and Electronic Engineering, Yonsei University, Seoul 03722, South Korea, and also with the CTO Division, SoC Center, SoC Fundamental Technology Laboratory, LG Electronics, Seoul 06772, South Korea.

Jongyoon Won and Youngcheol Chae are with the Department of Electrical and Electronic Engineering, Yonsei University, Seoul 03722, South Korea (e-mail: ychae@yonsei.ac.kr).

Byounghan Min, Dooyeoun Kim, Jihyun Kim, and Jeong-Hyu Yang are with the CTO Division, SoC Center, SoC Fundamental Technology Laboratory, LG Electronics, Seoul 06772, South Korea.

Digital Object Identifier 10.1109/TCSII.2025.3554297

TABLE I
INTERCONNECT PARASITICS WITH TECHNOLOGY SCALING

	40nm	28nm	16nm	7nm	5nm
C_P	1	0.95	0.83	1.05	1.07
R_P	1	1.5	6	20	63

Due to the limited voltage headroom, a cascade approach has become inevitable. In response, a multi-stage amplifier, such as a ring amplifier, has been developed that achieves high energy efficiency by exploiting slew-based charging [2], [3], [4], [5], [6]. Nevertheless, the multi-stage amplifier requires precise stabilization techniques to ensure stability within the closed-loop configuration. An alternative approach is to use open-loop dynamic or G_m-based amplifiers for the residue gain. These have been shown to achieve high energy efficiency by circumventing the stringent requirements on the closed-loop residue amplifiers [7], [8], [9], [10], [11]. However, the open-loop configuration is susceptible to variations in process, voltage, and temperature (PVT), requiring additional calibration that increases the design complexity.

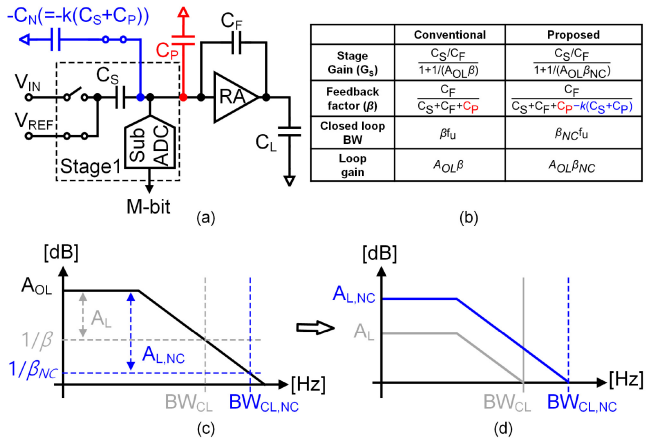
On the other hand, a feedback factor (β) compensation technique using a reference buffer or negative capacitor (NC) at the virtual ground of the residue amplifier has been proposed [12], [13]. This offers a clear advantage in terms of reducing the requirements on the main amplifier, resulting in an improved energy efficiency. However, the main amplifier of these ADCs still uses a high-performance amplifier with a telescopic OTA [13] or two-stage OTA [12]. In this brief, by using the dynamic NC for β compensation, a design optimization of the OTA is described to achieve an improved energy-efficiency. Implemented in a 6-nm FinFET process, a 10-bit 500-MS/s pipelined SAR ADC using a single-stage inverter-based OTA achieves 53.6 dB SNDR and 66.9 dB SFDR for Nyquist input, respectively, while consuming only 2.7 mW, resulting in a Walden figure-of-merit (FoM$_W$) of 13.8 fJ/conv.-step. The remaining of this brief is organized as follows. Section II describes the proposed pipelined SAR ADC, while Section III summarizes the measurement results. Finally, this brief ends with conclusions in Section IV.

II. Proposed Pipelined SAR ADC

A. β Compensation Technique

Fig. 1(a) presents the closed-loop residue amplification incorporating the β compensation technique [12]. During the amplification phase, the NC ($-C_N$) is connected to the virtual

1549-7747 © 2025 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.
Authorized licensed use limited to: Tamkang Univ.. Downloaded on February 02,2026 at 06:19:02 UTC from IEEE Xplore. Restrictions apply.



	Conventional	Proposed
Stage Gain (G_s)	$\frac{C_S/C_F}{1+1/(A_{OL}\beta)}$	$\frac{C_S/C_F}{1+1/(A_{OL}\beta_{NC})}$
Feedback factor (β)	$\frac{C_F}{C_S+C_F+C_P}$	$\frac{C_F}{C_S+C_F+C_P-k(C_S+C_P)}$
Closed loop BW	βf_u	$\beta_{NC} f_u$
Loop gain	$A_{OL}\beta$	$A_{OL}\beta_{NC}$

Fig. 1. (a) The closed-loop residue amplification with the proposed β compensation, (b) comparison of conventional and proposed scheme, Bode plot for (c) A_{OL} and (d) A_L.

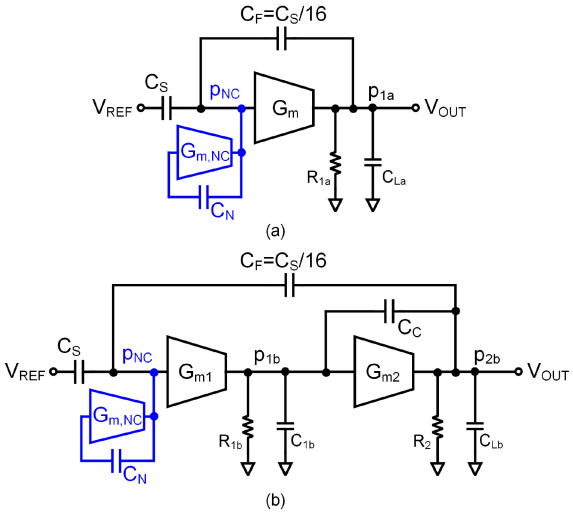


Fig. 2. Closed-loop NC-assisted residue amplifier ac model (a) 1-stage OTA and (b) 2-stage OTA.

ground to provide β compensation. This serves to mitigate the impact of C_P and C_S. As the compensation coefficient k approaches 1, ideal compensation is achieved ($\beta_{NC} = 1$). Fig. 1(b) illustrates that the β compensation enhances the loop gain (A_L) and the closed-loop bandwidth (BW_{CL}) residue amplifier by an x-factor of β_{NC}/β. Fig. 1(c) and (d) visualize the summary of (b) in the form of Bode plots for the A_{OL} and the adjusted A_L ($= A_{OL}\beta$).

B. Comparison of the NC-Assisted Residue Amplifiers

In prior work using a 28-nm CMOS process, the A_{OL} requirement could be reduced to 50 dB by applying β compensation technique and an inverter-based 2-stage OTA is used as a residue amplifier [12]. However, the use of 2-stage OTA requires additional compensation to ensure stability, which adds complexity to the design and limits its bandwidth, also resulting in higher power consumption. Fig. 2 depicts both the 1-stage and 2-stage OTAs with NC. The load capacitance (C_{La} and C_{Lb}) includes the parasitic capacitance resulting from metal routing associated with the residual amplifier,

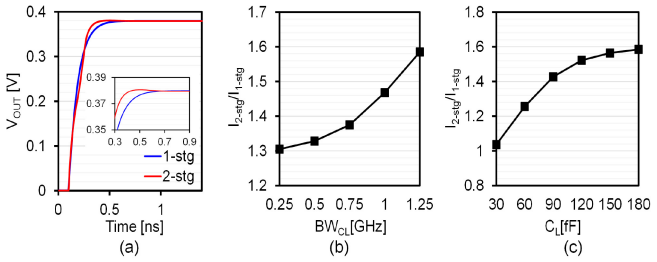


Fig. 3. (a) Step response of two amplifiers, simulated current consumption ratio of two amplifiers vs (b) BW_{CL} and (c) C_L.

	A_L	G_m @ 1-stg OTA (=G_m)	G_m @ 2-stg OTA (=G_{m1}+G_{m2})
w/o NC	25.4dB	26.2mS	34.1mS
w/ NC	37.5dB	8.0mS	15.6mS
X-factor		**3.3X**	**2.2X**

(a)

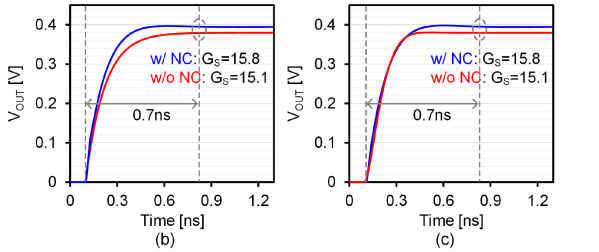


Fig. 4. Comparison of two schemes with NC (a) Summary of the simulated results, step response of (b) 1-stage OTA and (c) 2-stage OTA.

assuming C_{1b} to be one-third of C_{Lb}. Fig. 3(a) illustrates the step response of the two amplifiers, each with a similar A_{OL} (= 50 dB) and BW_{CL} (= 1.25 GHz). The absence of negative capacitance (NC) has resulted in an A_L of only 25 dB due to β, with the stage gain (G_S) failing to reach the desired 16×, thus resulting in a gain error.

A detailed examination of the zoomed-in waveform reveals that the 1-stage OTA exhibits a relatively improved stability. Fig. 3(b) shows that the current consumption of the 2-stage OTA increases compared to the 1-stage OTA as the required BW_{CL} rises. Fig. 3(c) indicates that the energy efficiency of the 2-stage OTA decreases with increasing C_L. Since the C_P in the FinFET process significantly enlarges the C_L, minimizing the number of amplifier stages is beneficial. Therefore, employing the 1-stage OTA as the main amplifier with NC to compensate for the deficient A_{OL} and BW_{CL} is a promising strategy to improve energy efficiency.

Fig. 4(a) depicts the A_L and required transconductance (G_m) for the combination of the NC and the two different OTAs. Although the two OTAs depicted in Fig. 3 necessitate a G_m of 26.2 mS and 34.1 mS, respectively, the A_L without the NC is limited to a 25.4 dB, and the G_S turns out to be insufficient (∼15.1). The NC-assisted OTA boosts the A_L to 37.5 dB, thereby achieving the resolution requirements for the subsequent stage (6 bits). The NC-assisted OTA achieves the required G_S (=15.8), while reducing the G_m of the main OTA to 8 and 15.6 mS, respectively. The requirements of the 1-stage and 2-stage OTAs can be reduced by 3.3× and 2.2×, respectively. As can be seen, the 1-stage OTA is 1.5 times more efficient than the 2-stage OTA. In the case of the 2-stage

Authorized licensed use limited to: Tamkang Univ.. Downloaded on February 02,2026 at 06:19:02 UTC from IEEE Xplore. Restrictions apply.

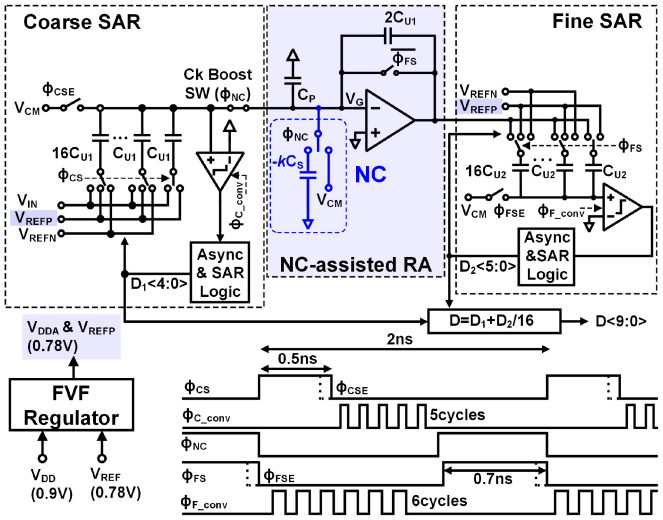


Fig. 5. Schematic and timing of proposed pipelined SAR ADC.

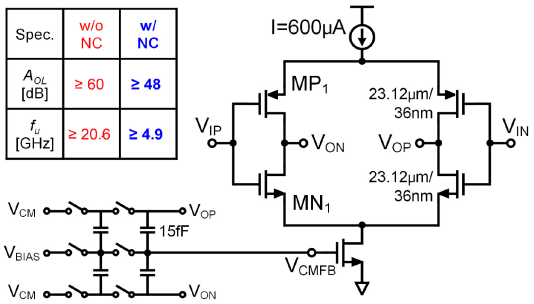


Fig. 6. Schematic of the main amplifier and its specification.

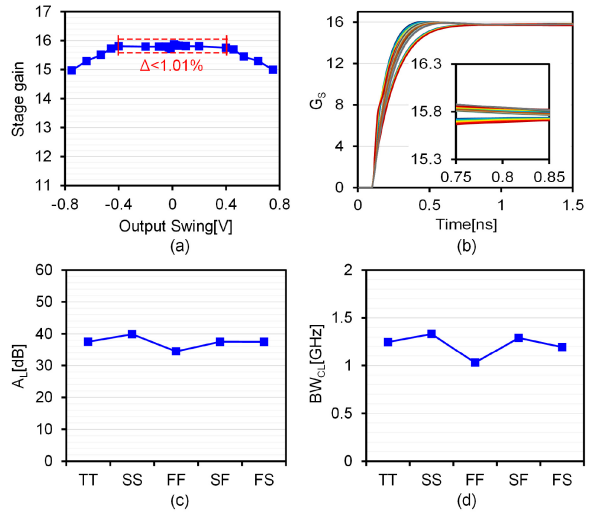


Fig. 7. Proposed dynamic NC assisted residue amplifier's simulation results, (a) Stage gain vs Output swing, (b) Step-response vs corner (TT, SS, FF, FS, SF) and temperature (0°C - 80°C), (c) A_L, and (d) BW_{CL}.

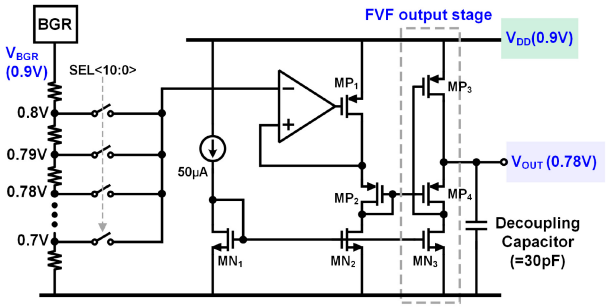


Fig. 8. Schematic of the on-chip regulator.

OTA, the pole introduced by NC (p_{NC}) is positioned between nodes p_{1b} and p_{2b}, additional current is required to drive p_{2b} at a high frequency to ensure stability.

C. ADC Implementation

Fig. 5 shows the schematic and timing diagram of the proposed pipelined SAR ADC. The C_{U1} of the 5-bit coarse SAR is 8.375 fF, and with a total sampling capacitance of 268 fF and a C_F of 16.75 fF, the G_S is 16×. The NC is of the dynamic type, as implemented in prior work [12]. The use of the 6-nm process introduces a significantly higher C_P at the virtual ground node compared to previous technology nodes, raising concerns about C_P variation and stability during β compensation. Consequently, the NC's compensation coefficient k is set to 0.8, with the NC designed to be 295 fF to compensate for 368 fF, which includes the C_P (≈100 fF) of the virtual ground node. This adjustment increases β from 0.059 to 0.24, relaxing the requirements for A_{OL} and f_u by a factor of 4.07. The inverter-based residue amplifier, shown in Fig. 6, must achieve the G_S of 16× within 700 ps, which requires the A_L of 36 dB and the BW_{CL} exceeding 1.2 GHz. The use of NC allows the required A_{OL} and f_u to be relaxed to 48 dB and 4.9 GHz, respectively. Fig. 7 presents the simulation results of the proposed dynamic NC-assisted residue amplifier. As shown in Fig. 7(a), it maintains approximately 16× gain with a variation of 1.01% within ±0.4 V differential output swing range. Fig. 7(b) illustrates the simulation results under temperature variations from 0 to 80°C across three process corners, which remains within a specified timing range within a maximum error of 0.5 LSB. The NC value shows minimal variation across the corners, with the highest value occurring at the SS corner and the lowest at the FF corner. Fig. 7(c) and 7(d) demonstrate that both the A_L and BW_{CL} are smallest at the FF corner due to the reduced NC value. Nevertheless, this reduction is not significant enough to pose any operational issues.

The 6-nm process exhibits significant C_P and R_P inherent to the implementation. In particular, the R_P in the lower metal layer is large enough to affect the performance of digital circuits composed of basic logic gates. To address this, the power supply is split, delivering 0.78 V to the analog section and 0.9 V to the digital section. Since the analog portion powers the residue amplifier and the V_{REFP}, a clean and stable voltage is required. To accomplish this, an on-chip flipped voltage follower (FVF) regulator is integrated. The 0.9 V supply powers both the FVF regulator and the digital components of the ADC, while the stabilized 0.78 V output is provided to the analog section. Fig. 8 shows the schematic of the on-chip regulator, which comprises a bias circuit, an error amplifier, and an FVF output stage [15]. The FVF output stage enables fast transient performance by leveraging a local feedback loop to reduce output impedance and enhance current

Authorized licensed use limited to: Tamkang Univ.. Downloaded on February 02,2026 at 06:19:02 UTC from IEEE Xplore. Restrictions apply.

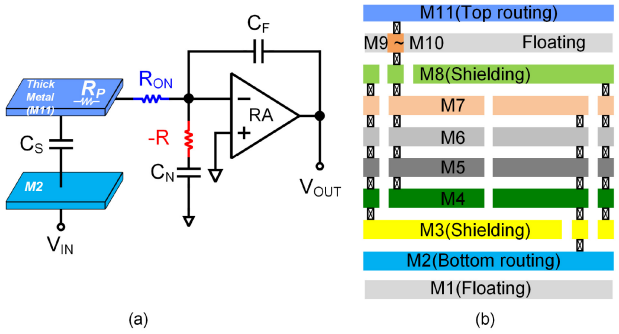


Fig. 9. (a) Routing of the C-DAC and Residue amplifier at virtual ground node and (b) encapsulated MOM capacitor layout and routing.

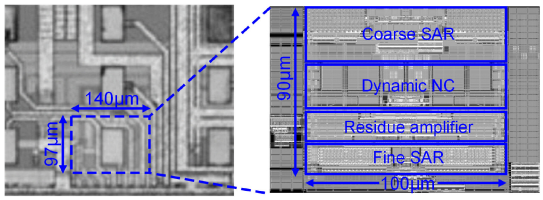


Fig. 10. Chip micrograph.

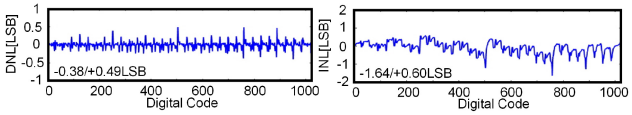


Fig. 11. Measured DNL/INL.

sourcing capability. A 30 pF decoupling capacitor is placed in the available idle space to ensure stable operation.

As shown in Fig. 9(a), the R_P of the virtual ground node must be minimized, as it can induce fluctuations in the R_{ON} used to compensate for the negative resistance (-R) of the NC. Fig. 9(b) illustrates the encapsulated MOM capacitor and its routing for the unit capacitor, which simplifies the application of the concept in Fig. 9(a). The unit capacitance is formed using metal layers M4 through M7, with bottom routing to M2 and top routing to M11. To reduce C_P between the bottom plate (M4) and routing (M2), M3 is used as a shield, while the lower metal layers M1 and M0 are left unused and floating, as lower layers typically exhibit higher R_P. To minimize the R_P of the top plate, thick metal (M11) is employed, effectively reducing R_P but at the cost of increased C_P. To address this, shielding is provided using M8, and layers M9 through M10 are left floating. Additionally, the capacitors are encapsulated by M4 through M7 to reduce capacitive coupling between adjacent capacitors. All shielding layers are grounded to maintain a stable potential.

III. Measurement Results

The proposed 10-bit 500-MS/s ADC prototype is fabricated in a 6-nm FinFET process. It occupies an active area of 0.014 mm^2, as shown in Fig. 10, where an active area excluding the integrated regulator and decoupling capacitor is only 0.009 mm^2. As shown in Fig. 11, the measured differential nonlinearity (DNL) and integral nonlinearity (INL) are −0.38/+0.49 and −1.64/+0.60, respectively. Fig. 12 shows

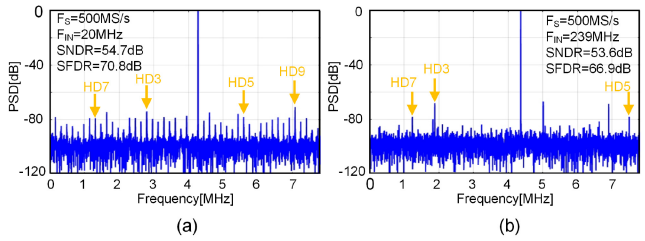


Fig. 12. Measured output spectrum with (a) 20-MHz, (b) 239-MHz.

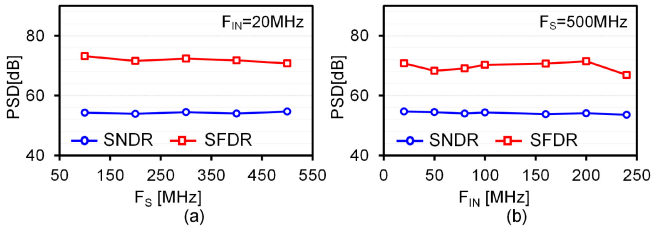


Fig. 13. Measured SNDR/SFDR vs (a) F_S, (b) F_{IN}.

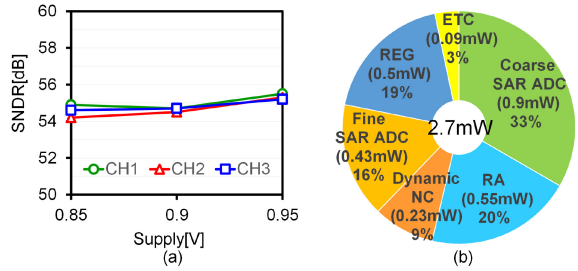


Fig. 14. (a) Measured SNDR with supply and chip variations and (b) power breakdown.

the measured power spectral densities (PSDs) decimated by a factor of 32. For the 20-MHz input, the SNDR and SFDR are 54.7 dB and 70.8 dB, respectively. For the near Nyquist input of 239-MHz, the measured SNDR and SFDR are 53.6 dB and 66.9 dB, respectively. Fig.13(a) shows the measured performance for different sampling frequencies at the 20-MHz input, where the variations of SNDR and SFDR are in the range of 0.8 dB and 2.4 dB, respectively. Fig. 13(b) demonstrates the measured performance for different F_{IN} at a sampling rate of 500-MS/s, which confirms that the performance remains stable up to the Nyquist frequency. By using the integrated regulator, the measured results for the supply variations of the three chips remain stable, as shown in Fig. 14(a). Fig. 14(b) shows the power breakdown of the ADC. The overall power consumption is only 2.7 mW. By using only 9% of the total power consumption with the dynamic NC, the power consumption of the residue amplifier can be reduced to 0.55 mW (∼20%).

Table II provides a performance summary and comparison with prior state-of-the-art works fabricated in a 28-nm process [12], or below [3], [5], [6], [14]. The use of an external reference regulator further enhances the FoM_W (= 9.6 fJ/conv.-step), as eliminating the power consumption of the integrated regulator and the use of a low-noise external regulator (LT3045) allow an improvement in SNDR. Compared to [12], this brief achieves both a better FoM_W

Authorized licensed use limited to: Tamkang Univ.. Downloaded on February 02,2026 at 06:19:02 UTC from IEEE Xplore. Restrictions apply.

TABLE II
PERFORMANCE SUMMARY AND COMPARISON WITH STATE-OF-THE-ART WORKS

	This work		A-SSCC' 23 Kwon [12]	ISSCC' 21 Baek [14]	ISSCC' 24 Lim [6]	ISSCC' 19 Hershberg [3]	JSSC' 22 Lagos [5]	
Process[nm]	**6**		28	7	8	16	16	
Architecture	**Pipelined SAR**		Pipelined SAR	SAR + TI-DSM	Pipelined SAR	Pipeline	Pipelined SAR	
Residue Amplifier	**Inverter-based 1-stage OTA**		Inverter-based 2-stage OTA	No	Ring Amplifier	Ring Amplifier	Ring Amplifier	
Amplifier Auxiliary technique	**NC-assisted β Compensation**		NC-assisted β Compensation	Residue path Replica	Dynamic bias	Gain calibration	Dither-based Gain calibration	
Resolution[bits]	**10**		11	12	12	11	12	
F_S[MS/s]	**500**		360	600	400	600	500	
Regulator	**Yes**	**No**	No	No	No	No	Yes	No
Area[mm^2]	**0.014**	**0.009**	0.016	0.037	0.017	0.037	0.047	0.015
Supply[V]	**0.9**	**0.78(A)/0.9(D)**	0.95	0.8	0.9	0.85	0.9	
Power[mW]	**2.7**	**2.1**	3.9	13	2.1	6	3.3	2.8
SNDR[dB]	**53.6**	**54.5**	58	55.3	62.8	60.2	62.3	62.9
SFDR[dB]	**66.9**	**68.3**	77.9	67.8	68.6	78.3	75.5	75.5
*FoM_W[fJ/c.-s]	**13.8**	**9.6**	16.7	45.6	4.6	12.0	6.2	4.9

*FoM_W = Power/($2^{ENOB}\times 2\times$BW)

with a higher operation speed (1.38×). Compared to an ADC that does not use a residue amplifier in a 7-nm FinFET [14], this brief achieves significantly improved energy efficiency at similar resolution and speed. Compared to ADCs with FinFET process that use a ring amplifier [3], [5], [6], this brief achieves competitive energy efficiency and operation speed.

IV. CONCLUSION

This brief presents a pipelined SAR ADC featuring a dynamic NC-assisted residue amplifier fabricated in a 6-nm FinFET process. The dynamic NC enables high energy efficiency by exploiting a single-stage inverter-based OTA, required for low supply voltage operation. The ADC prototype achieves a competitive FoM_W of 13.8 fJ/conv.-step when operating at 500-MS/s, and is robust against PVT variations due to its closed-loop configuration. The combination of a single-stage OTA with a dynamic NC represents a promising solution for energy-efficient pipelined SAR ADCs in advanced process nodes.

REFERENCES

[1] S. Goyal et al., "Design challenges and techniques for 5nm FinFET CMOS analog/mixed-signal circuits," in *Proc. Int. Conf. VLSI Design (VLSID)*, 2023, pp. 98–103.

[2] B. Hershberg, S. Weaver, K. Sobue, S. Takeuchi, K. Hamashita, and U.-K. Moon, "Ring amplifiers for switched capacitor circuits," *IEEE J. Solid-State Circuits*, vol. 47, no. 12, pp. 2928–2942, Dec. 2012.

[3] B. Hershberg et al., "A 6-to-600MS/s fully dynamic ringamp pipelined ADC with asynchronous event-driven clocking in 16nm," in *Proc. IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, Feb. 2019, pp. 68–70.

[4] J.-S. Park, T.-J. An, G.-C. Ahn, and S.-H. Lee, "A 12.1 fJ/conv.-step 12b 140 MS/s 28-nm CMOS pipelined SAR ADC based on energy-efficient switching and shared ring amplifier," *IEEE Trans. Circuits Syst. II, Exp. Briefs*, vol. 66, no. 7, pp. 1119–1123, Jul. 2019.

[5] J. Lagos et al., "A 10.1-ENOB, 6.2-fJ/conv.-step, 500-MS/s, ringamp-based pipelined-SAR ADC with background calibration and dynamic reference regulation in 16-nm CMOS," *IEEE J. Solid-State Circuits*, vol. 57, no. 4, pp. 1112–1124, Apr. 2022.

[6] Y. Lim et al., "A 2.08 mW 64.4 dB SNDR 400MS/s 12b pipelined-SAR ADC using mismatch and PVT variation tolerant dynamically biased ring amplifier in 8nm," in *Proc. IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, Feb. 2024, pp. 170–172.

[7] H. Yoon, C. Lee, T. Kim, Y. Kwon, and Y. Chae, "A 65-dB-SNDR pipelined SAR ADC using PVT-robust capacitively degenerated dynamic amplifier," *IEEE J. Solid-State Circuits*, vol. 58, no. 4, pp. 961–971, Apr. 2023.

[8] J. Hao, Y. Shen, J. Zhang, Y. Zhang, S. Liu, and Z. Zhu, "A 14b 180MS/s pipeline-SAR ADC with adaptive-region-selection technique and gain error calibration," *IEEE Trans. Circuits Syst. II, Exp. Briefs*, vol. 71, no. 1, pp. 16–20, Jan. 2024.

[9] S. Liu, H. Han, Y. Shen, and Z. Zhu, "A 12-bit 100-MS/s pipelined-SAR ADC with PVT-insensitive and gain-folding dynamic amplifier," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 67, no. 8, pp. 2602–2611, Aug. 2020.

[10] J. Zhang, X. Ren, S. Liu, C.-H. Chan, and Z. Zhu, "An 11-bit 100-MS/s pipelined-SAR ADC reusing PVT-Stabilized dynamic comparator in 65-nm CMOS," *IEEE Trans. Circuits Syst. II, Exp. Briefs*, vol. 67, no. 7, pp. 1174–1178, Jul. 2020.

[11] K.-J. Moon, D.-S. Jo, W. Kim, M. Choi, H.-J. Ko, and S.-T. Ryu, "A 9.1-ENOB 6-mW 10-bit 500-MS/s pipelined-SAR ADC with current-mode residue processing in 28-nm CMOS," *IEEE J. Solid-State Circuits*, vol. 54, no. 9, pp. 2532–2542, Sep. 2019.

[12] Y. Kwon, S. Lee, C. Lee, H. Yoon, B. Min, and Y. Chae, "An 11bit 360MS/s pipelined SAR ADC with dynamic negative-C assisted residue amplifier," in *Proc. IEEE Asian Solid-State Circuits Conf. (A-SSCC)*, Nov. 2023, pp. 1–3.

[13] H. H. Boo, D. S. Boning, and H.-S. Lee, "A 12b 250 MS/s pipelined ADC with virtual ground reference buffers," *IEEE J. Solid-State Circuits*, vol. 50, no. 12, pp. 2912–2921, Dec. 2015.

[14] S. Baek et al., "A 12b 600MS/s pipelined SAR and 2x-interleaved incremental delta-sigma ADC with source-follower-based residue-transfer scheme in 7nm FinFET," in *Proc. IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, 2021, pp. 172–174.

[15] Y. Shen, X. Tang, X. Xin, S. Liu, Z. Zhu, and N. Sun, "A 10-bit 100-MS/s SAR ADC with always-on reference ripple cancellation," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 69, no. 10, pp. 3965–3975, Oct. 2022.

Authorized licensed use limited to: Tamkang Univ.. Downloaded on February 02,2026 at 06:19:02 UTC from IEEE Xplore. Restrictions apply.

Hybrid Integration of Gate-All-Around Stacked Si Nanosheet FET and Si/SiGe Super-Lattice FinFET to Optimize 6T-SRAM for N3 Node and Beyond

Xuexiang Zhang, Jiaxin Yao, *Associate Member, IEEE*, Yanna Luo, Lei Cao, *Graduate Student Member, IEEE*, Yantong Zheng, Qingzhu Zhang, *Associate Member, IEEE*, Zhenhua Wu, *Member, IEEE*, and Huaxiang Yin, *Senior Member, IEEE*

***Abstract*— The implementation of vertically stacked gate-all-around nanosheet FET (GAA NSFET) may help improve the performance of static random access memory (SRAM) for the design flexibility with variable NS widths. However, the method that often relies on the increasing of device width for higher driving current is to cause an increased SRAM cell area and degrade the scalability of transistor in advanced nodes. In this article, an innovated SRAM bitcell design with hybrid integration of Si NSFET and Si/SiGe super-lattice FinFET (SL-FinFET) in one SRAM cell is proposed for less area and improved performance in 3 nm and beyond nodes. Under the same footprint, p-type SL-FinFET delivers obviously enhanced driving current compared with NSFET and it may yield improved circuit performance for the SRAM cell with natural transistor ratio. The integration process method and the layout schemes for two kinds of typical 6T-SRAM cells, including the high-robustness cell (HI-R Cell) and the high-speed cell (HI-S Cell) with the hybrid integration method, are presented for optimizing high-density (HD) and high-performance (HP) SRAM. As compared with the HD NSFET SRAM cell, the HD HI-R Cell achieves 16.2% improvement in read static noise margin (RSNM) and the HD HI-S Cell achieves 21.6% reduction in read time and 28.6% reduction in write time. The HP hybrid integrated cell even pushes the SRAM cell performance arriving to the theoretical limits of regular NSFETs. Moreover, the hybrid integration scheme may provide another revolutionary method to optimize transistor ratio by adjusting the height of SiGe thickness in the transistors.**

***Index Terms*— 3-nm process node, 6T static random access memory (6T-SRAM), hybrid integration, Si/SiGe super-lattice FinFET (SL-FinFET), vertically stacked gate-all-around nanosheet FET (GAA NSFET).**

I. Introduction

LARGE volume and fast static random access memory (SRAM) help the computing system boost performance and occupy a considerable portion of the die size in many computing chips. Therefore, the development of higher density SRAM with less bitcell area as well as stronger driving ability inside is one of the most critical tasks to maintain the scaling of CMOS technology in past decades [1]. The bitcell area of standard 6T-SRAM has already shrunk to 0.021 μm^2 at 5 nm node for updated FinFET process [2].However for 3 nm or beyond nodes, the gate-all-around nanosheet FETs (GAA NSFETs) have proven to be a promising successor of the FinFET, with better electrostatic in channel and stronger device driving performance due to superior gate control capability on vertically stacked nanosheet channels [3], [4], [5], [6], [7], [8]. In addition, GAA NSFET offers the design flexibility that was sacrificed partly while FinFETs replace traditional planar transistors [9]. For high-density (HD) SRAM bitcells, FinFET must adopt 1:1:1 or 1:1:2 fin configurations with the quantized width design limitations, which often results in the degradation of bitcell performance, such as the conflict of read and write stability [10]. The novel NSFET can obtain 1: α: β configuration to achieve improved bitcell specifications, such as the increasing of the disturb margin [11].

A series of device, process, and design optimization technologies for new NSFETs building HD and high-performance

Manuscript received 9 November 2023; revised 23 December 2023; accepted 18 January 2024. Date of publication 2 February 2024; date of current version 1 March 2024. This work was supported in part by the Strategic Priority Research Program of Chinese Academy of Sciences under Grant XDA0330302; in part by the Beijing Natural Science Foundation under Grant 4224096; in part by the Project funded by China Postdoctoral Sciences Foundation under Grant 2021M703443; in part by the Youth Innovation Promotion Association, Chinese Academy of Sciences under Grant Y9YQ01R004; in part by the Science and Technology program of Beijing Municipal Science and Technology Commission under Grant Z201100006820084; and in part by NSFC under Grant 12174423. The review of this article was arranged by Editor N. Xu. *(Corresponding authors: Qingzhu Zhang; Zhenhua Wu; Huaxiang Yin.)*

The authors are with the Integrated Circuit Advanced Process R&D Center and State Key Laboratory of Fabrication Technologies for Integrated Circuits, Institute of Microelectronics, Chinese Academy of Sciences, Beijing 100029, China, and also with the School of Integrated Circuits, University of Chinese Academy of Sciences, Beijing 100049, China (e-mail: zhangqingzhu@ime.ac.cn; wuzhenhua@ime.ac.cn; yinhuaxiang@ime.ac.cn).

Color versions of one or more figures in this article are available at https://doi.org/10.1109/TED.2024.3358251.

Digital Object Identifier 10.1109/TED.2024.3358251

0018-9383 © 2024 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: Tamkang Univ.. Downloaded on February 04,2026 at 07:12:59 UTC from IEEE Xplore. Restrictions apply.

(HP) SRAM bitcells and circuits are reported [12], [13], [14]. Huynh-Bao et al. [15], [16] proposed device threshold voltage retargeting technology to reduce the minimum operating voltage (V_{min}) of SRAMs with NSFETs. Vashishtha [17] have done a lot of research on the optimization of GAA NSFET parameters on SRAM performance. However, it is also found that a larger parasitic capacitance in GAA NSFETs introduced by its more complex 3-D channel structure makes a longer read and write delay in SRAM bitcell [18], [19], [20], [21], [22], [23]. Especially for HD SRAM with 1:1:1 configuration with extremely scaled nodes, the flexible increase of nanosheets width may improve read and write margin or driving ability but obviously increase the cell area, which perhaps awakes a great challenge for the NSFET competing with the cutting-edge FinFET technology [24].

In order to obtain the best SRAM bitcell performance and the most aggressively scaling area with GAA NSFETs at the same time, an innovative approach by hybrid integrating Si NSFET with Si/SiGe super-lattice FET (SL-FinFET) in 6T-SRAM cell is firstly proposed in the article. The SL-FinFET was formed by no nanosheets release process implemented in replacement metal-gate (RMG) [25] and applied into the hybrid integration of high-voltage I/O devices with NSFET core devices [26] to overcome the challenge of thick gate insulator filling between stacked Si NSs. The stronger driving ability [27], [28], [29], [30] than NSFET will reduce the area cost of obtaining greater driving strength and cope with the high drive requirements.

In the article, at the first time, the hybrid integration scheme for combining Si NSFET with Si/SiGe SL-FinFET in one 6T-SRAM bitcell (HI-SRAM) is proposed, which may effectively take both advantages of these two devices to optimize the critical circuit performance at scaled process nodes. The integration flow totally compatible with mainstream NSFETs manufacturing process is regulated and the electrical characteristics of new SRAM bitcell at 3-nm node are carefully evaluated by technical computer-aided design (TCAD) and custom circuit emulation tool (SPICE). With the revolutionary structure and optimized parameters, the speed and stability have been obviously improved. The results indicated that it is an effective approach to further boost the gain of power, performance, and area (PPA) in future circuits by GAA NSFET processes.

II. Hybrid Integration Method for SRAM

A. Hybrid Integration Scheme for SRAM

In general custom-integrated circuit (IC) design, the regulations of channel widths in transistors are critical for circuit performance optimizations. For the 6T-SRAM bitcell design, it often requires a ratio of pull-down (PD) transistor width (W_{PD}) > access transistor width (W_{AX}) > pull-up (PU) transistor width (W_{PU}) for better performance [31]. With the guide of the design method, in Si NSFET SRAM cell as shown in Fig. 1(a), the value of W_{PD}: W_{AX}: W_{PU} is flexibly adjusted and fixed at 1: α: β configuration, which may cause a slight increasing of cell area as compared with the natural design with W_{PD}: W_{AX}: W_{PU} = 1:1:1. In the general NSFET

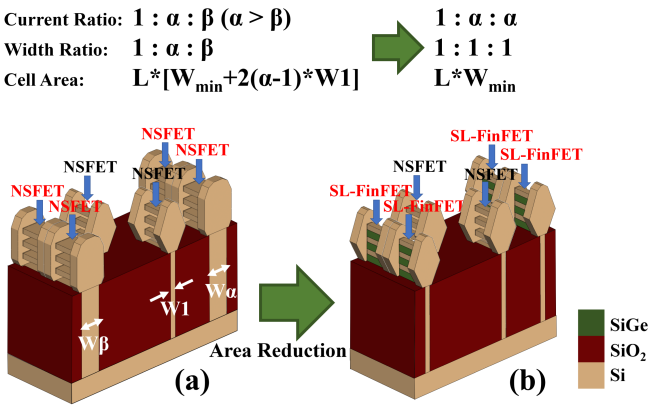


Fig. 1. Bitcell scheme for (a) NSFET SRAM and (b) hybrid integrated SRAM.

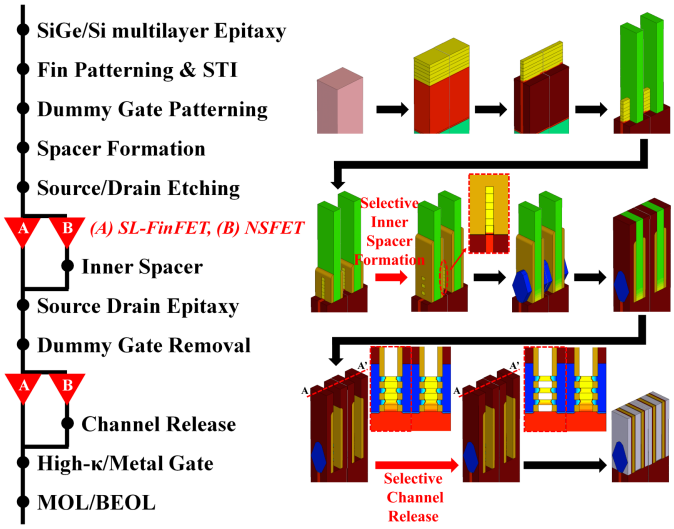


Fig. 2. Designed process flow of hybrid integration. Different from traditional NSFET, mainstream processes are marked with red arrows.

manufacturing process, the SL-FinFETs are easily formed by avoiding the nanosheets release process with protective mask and provide higher driving current. With different transistor driving ability between the NSFET and the SL-FinFET, the proposed hybrid integration of these two kinds of devices in SRAM cell is expected to provide the desired performance by following the guide of $W_{PD} > W_{AX} > W_{PU}$ with the natural design of W_{PD}: W_{AX}: W_{PU} = 1:1:1 [see Fig. 1(b)] for obtaining the minimum cell area in high-density SRAM circuits.

B. Process of Hybrid Integration

The designed process flow of hybrid integration of the GAA NSFET and the SL-FinFET on one Si wafer is shown in Fig. 2. At first, the epitaxy of the stacked SiGe/Si layers is grown on the bulk Si substrate, where the thickness of the Si layer is defined as the single-layer nanosheet channel thickness of the NSFET, and the total thickness of the SiGe/Si layers is the Fin channel height of the SL-FinFET. Then, fins, shallow trench isolation (STI), dummy gate, and spacer are formed in sequence with general GAA NSFET processes [32]. In the next steps, the source/drain (S/D) is etched by anisotropic methods

Authorized licensed use limited to: Tamkang Univ.. Downloaded on February 04,2026 at 07:12:59 UTC from IEEE Xplore. Restrictions apply.

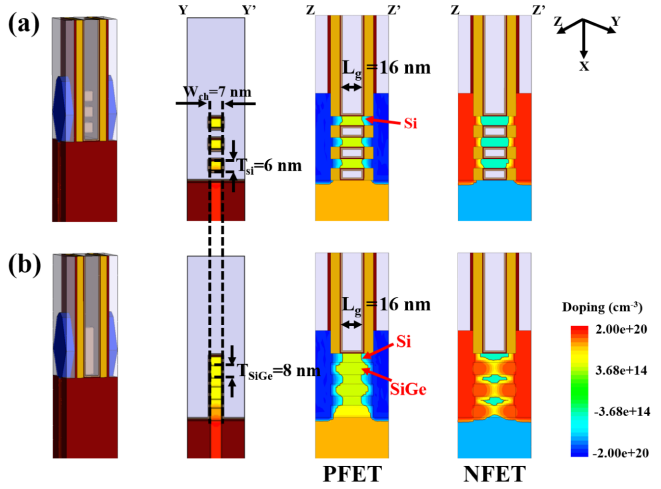


Fig. 3. Structure of the simulation domain of the 3-nm node (a) NSFETs and (b) SL-FinFETs.

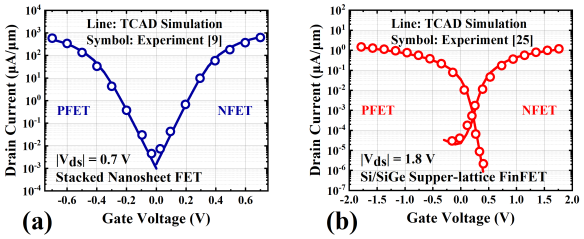


Fig. 4. I_D–V_G calibration of (a) stacked Si NSFET and (b) Si/SiGe super-lattice FinFET.

to the Si substrate and a shadowing mask is used to protect the SiGe channels in the SL-FinFET region, while the SiGe in the NSFET area is etched back and the conventional inner spacer is formed. After the removal of the shadowing mask, the embedded selective SiGe or SiC epitaxy with p-type and n-type in-suit doping is applied to form the S/D regions of FETs. Then, the zero-level isolation layer (ILD-0) is deposited and planarized were performed to expose the dummy gate, followed by the selective removal of the dummy gate stack (poly-Si/gate oxide). For the second time, the shadowing mask is formed in the SL-FinFET region to protect the SL-Fin structure again. Next, the selective etching of SiGe in the NSFET region is performed to release the Si nanosheet channel. Finally, the shadowing mask is removed and the RMG with multilayer HKMG process as well as the transistor contacts and the back-end-of-line (BEOL) process followed the conventional process used for traditional NSFETs, are adopted for the formation of devices and interconnections in SRAM circuits.

This designed process is easy to realize a hybrid integration of SL-FinFET on conventional GAA NSFET flow only with twice additional shadowing mask processes, which is the modification of the conventional NSFET manufacture process only to a small extent and a little extra production cost. Moreover, the effective width of SL-FinFET can be adjusted by changing the SiGe thickness in the Si/SiGe epitaxy process, without altering the effective width of NSFET, which provide another method to adjust the effective transistor width without the increase of designed channel width for regulation of the ratio between W_{PD}, W_{AX}, and W_{PU}.

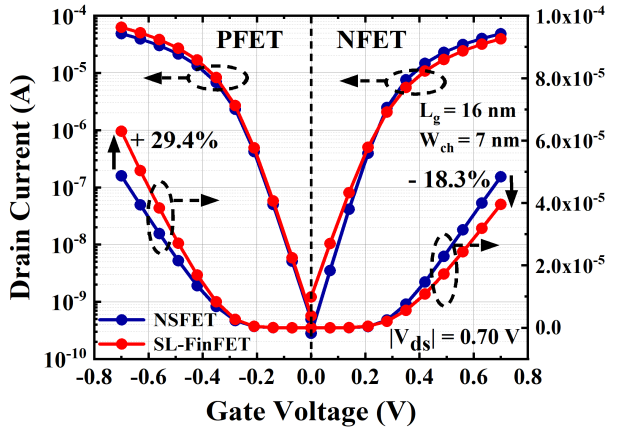
III. Device Structure and Performance

Fig. 3 shows the schematic of 3-nm-node NSFETs and SL-FinFETs formed by the designed process flow. PFET adopts SiGe S/D and NFET adopts SiC S/D. The main geometrical and process parameters for the NSFET are referred to the specification defined in International Roadmap for Devices and Systems (IRDS) 2022 [33]. Both NSFETs and SL-FinFETs are with the gate length (L_g) of 16 nm, channel projection width (W_{ch}) of 7 nm, and effective gate oxide thickness (EOT) of 0.808 nm. The thickness of the Si channel (T_{Si}) is 6 nm, and the thickness of the SiGe channel (T_{SiGe}) is 8 nm. There is a three-layer vertically stacked Si NS channel in one NSFET. Therefore, the total fin height of SL-Fin is 42 nm. The Ge concentration of SiGe in SL-Fin channels is set to 50% to significantly improve the mobility of the carriers. Besides, the NSFETs achieve (100) channel surface orientation and obtain higher electron mobility, while the hybrid integrated SL-FinFETs on the same wafer achieve (110) channel surface orientation and obtain higher hole mobility.

The electrical characteristics of both 3-nm-node Si NSFET and Si/SiGe SL-FinFET devices are simulated by Synopsys Sentaurus TCAD. The process simulation has considered the Ge mole fraction and the Ge intermixing caused by thermal annealing. The common drift-diffusion model solved by the Poisson and carrier continuity equation is adopted. The quantum effect, bandgap narrowing, band-to-band tunneling, carrier generation, and recombination are considered. To simulate the mobility degradation, the remote phonon and coulomb scattering at the channel and insulator interface are calculated, and the inversion and accumulation layer mobility models are used to calculate impurity, phonon, and surface roughness scatterings. At the same time, the velocity saturation model is implemented for high gate and drain bias. In addition, the Piezo model is introduced due to the lattice interaction between Si and SiGe [34]. Advanced calibrations of Si and SiGe material parameters [27], [35], [36], [37] have been performed for the unique super-lattice channel of the SL-FinFET. The TCAD simulation results for both NSFETs and SL-FinFETs are well calibrated with the experimental data of stacked nanosheet GAA transistors [9], [33] and Si/SiGe super-lattice FinFETs [25], as shown in Fig. 4, which ensures the simulation accuracy of the devices.

The operating voltage of simulated devices is fixed at 0.7 V. The thresholds of NFET and PFET are tuned to symmetrical 0.22 and −0.22 V, respectively. The simulation results showed a significant performance improvement in p-type SL-FinFET compared with p-type NSFET, particularly in driving ability, as shown in Fig. 5, and obtaining nearly 30% higher than that of the p-type NSFET. The significant driving current improvement is attributed to the enhanced hole mobility due to the (110) channel surface orientation of the SL-FinFET, the remained SiGe channel between Si NSs, and the compressive strain introduced by the superlattice stack structure in SL-FinFET. However, the short-channel effect of p-type SL-FinFET has slightly increased, resulting in degradation in subthreshold swing (SS),

Authorized licensed use limited to: Tamkang Univ.. Downloaded on February 04,2026 at 07:12:59 UTC from IEEE Xplore. Restrictions apply.



	p NSFET	p SL-FinFET	n NSFET	n SL-FinFET
SS (mV/dec)	69.2	70.0	64.1	75.4
DIBL (mV/V)	30.8	50.8	23.1	73.8

Fig. 5. Transfer characteristics of the 3-nm node NSFETs and SL-FinFETs at 0.7 V operating voltage.

drain-induced barrier lowering (DIBL), and OFF-current (Ioff). For NFET, transitioning from the GAA to the triple gate weakens the gate control ability, and the rapid diffusion of S/D doping in the channel significantly exacerbates the short-channel effect of the n-type SL-FinFET. So, it can be found that the performance of n-type SL-FinFET in all aspects such as ON-current (Ion), OFF-current (Ioff), SS, and DIBL is degraded. Therefore, the optimization to the SRAM cell with hybrid integration mainly focuses on the combined use of NSFET and p-type SL-FinFET in the SRAM circuit design.

IV. Hybrid Integrated SRAM Cell Design

In general, the 6T-SRAM cell consists of two PD NFETs and two PU PFETs to form two interlocked CMOS inverters for bit information storage, where two access (AX) transistors controlled by the word line (WL) and connected to complementary bit lines (BLs) are also included for bit information read/write control. In standard SRAM with GAA NSFET technology, NFETs are typically employed as AXs due to their higher driving current compared to PFETs under the same footprint. The n-type AX SRAM cell exhibits greater read stability with a larger PD ratio (the driving ability ratio of PD to AX) and higher write stability with a smaller PU ratio (the driving ability ratio of PU to AX). In the proposed SRAMs with hybrid integration, directly replacing p-type NSFETs with high-performance p-type SL-FinFETs as PU FETs may cause abnormal writing functions due to the increased PU ratio. Therefore, the design of p-type AX, which was also reported in earlier SRAM technology [38], was adopted instead. The incorporation of Si/SiGe SL-FinFET technology helps overcome the performance limitations of PFETs. In contrast to the n-type AX SRAM cell, the p-type AX SRAM cell may obtain higher read stability with a larger PU ratio and greater write stability with a smaller PD ratio.

Various hybrid integrated SRAM cells have been developed for different applications. One of the primary functions of SRAM is data exchange with CPU registers, and its speed directly impacts the maximum computing speed of the CPU. The driving strength of the AX plays a crucial role in determining the speed of SRAM. In light of this, a high-speed cell (HI-S Cell) has been designed, utilizing the hybrid integration method and employing p-type AXs, as shown in Fig. 6(b). To achieve enhanced speed, the HI-S Cell utilizes p-type SL-FinFET with high driving current as the AX. Additionally, to ensure proper read operations, p-type SL-FinFET is also employed as the PU, maintaining an optimal PU ratio. Furthermore, n-type NSFET is utilized as the PD to minimize leakage.

Moreover, certain SRAMs may encounter complex usage scenarios with high levels of external noise, demanding strong anti-interference capabilities. To address this requirement, a high-robustness cell (HI-R Cell) has been designed employing p-type AXs, as shown in Fig. 6(c). In this design, the AX utilizes p-type NSFET, while the PU employs p-type SL-FinFET. By leveraging the larger driving capability of the p-type SL-FinFET compared to the p-type NSFET, the PU ratio has been significantly enhanced, resulting in improved read stability.

The technology assumptions for the 3-nm-node SRAM are contacted gate pitch (CGP) = 45 nm, metal pitch (MP) = 30 nm, and channel spacer (CS) = 38 nm. The minimum channel width for both the NSFET and SL-FinFET is 7 nm, and the width of the nanosheets can vary within the range of 7–29 nm. SRAM cells are typically classified into HD cells and HP cells, depending on their intended usage. To achieve the most compact transistor arrangement and occupy the smallest cell area, HD SRAM cells employ the narrowest channel width. All three types of HD SRAM cells are compatible with the same manufacturing process, resulting in a consistent HD cell area of 90 × 180 nm^2, as shown in Fig. 6. Conversely, HP SRAM cells are designed to prioritize performance, even at the expense of increased area. In the HP NSFET SRAM design, the NSFET can be configured in 1: α: β setup, allowing for more flexible performance regulation. In the HP HI-SRAM design, the SL-FinFET follows a quantized width design, resulting in a fixed HP HI-SRAM cell area of 90 × 224 nm^2. The HP HI-S Cell aims to maximize the AX width to further enhance speed, while ensuring that the PU width remains consistent with the AX width to maintain normal read operation. The HP HI-R Cell is designed to maximize the PU ratio to further improve read stability.

V. Results and Discussions

The comparison and evaluation of the transient and static characteristics of the SRAM cells were performed through SPICE simulation. Compact models were created by fitting the physical parameters in the Berkeley short-channel insulated-gate field effect transistor model–common multigate (BSIM-CMG). Hold static noise margin (HSNM) and read static noise margin (RSNM) were extracted from the maximum inscribed square of each butterfly curve in the hold and read states. Additionally, write static noise margin (WSNM) was determined as the width of the minimum square that could be embedded between the lower-right half of the curves in the write states [39]. Read time (RT) is defined as the delay

Authorized licensed use limited to: Tamkang Univ.. Downloaded on February 04,2026 at 07:12:59 UTC from IEEE Xplore. Restrictions apply.

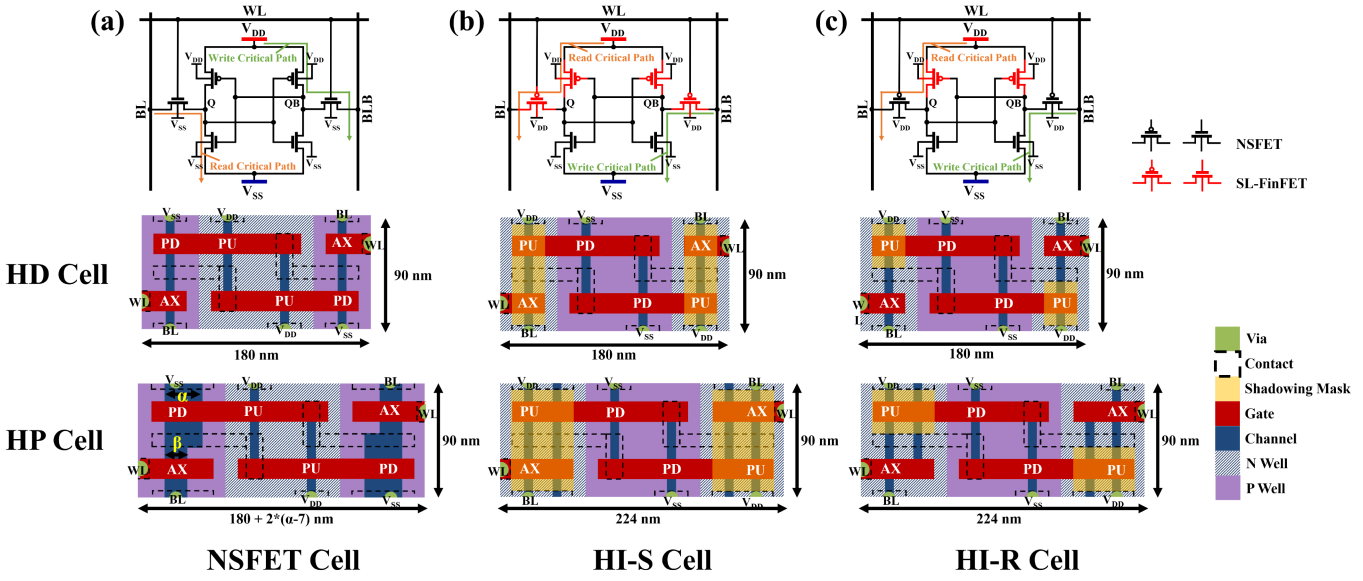


Fig. 6. Circuit structure and layout design of (a) NSFET SRAM cell, (b) hybrid integrated high-speed SRAM cell, and (c) hybrid integrated robust SRAM cell. The HD cell area for all types of SRAM cells is 90 × 180 nm.

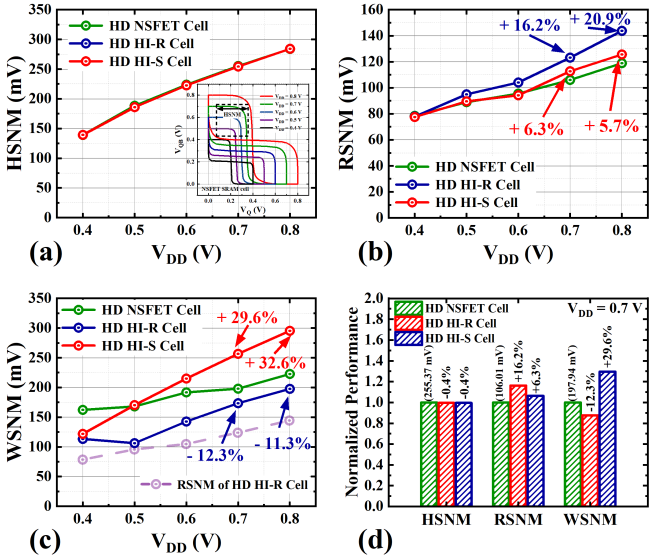


Fig. 7. Variation of (a) HSNM, (b) RSNM, and (c) WSNM of HD 6T-SRAM cells with V_{DD}. (d) Normalized performance of HD 6T-SRAM cells at 0.7 V operating voltage. The HI-R Cell balances the stability of reading and writing well.

between 50% WL signal rising/dropping edge (corresponding to the n-type and p-type AX SRAM cells, respectively) and 90% V_{DD} drop/rise of BLs. Similarly, write time (WT) is defined as the delay between 50% WL signal rising/dropping edge and 90% V_{DD} drop/rise of memory node.

A. HD SRAM Optimization

Fig. 7 shows the HSNM, RSNM and WSNM of the HD SRAM Cells. When the SRAM cell is in holding state, the AX is cutoff and the bi-stable circuit composed of two inverters is insensitive to the transistor ratio. As a result, the HSNM of the three SRAM cells remains largely consistent [see Fig. 7(a)]. The RSNM of the three HD SRAM cells exhibits a notable disparity, as shown in Fig. 7(b). In comparison to the HD NSFET Cell, the HD HI-R Cell demonstrates a noteworthy 16.2% improvement in RSNM at the operating voltage of 0.7 V. This improvement is attributed to the utilization of different types of devices for the PU and AX in the HD HI-R Cell. Consequently, the PU ratio increases to 63:48, surpassing the PD ratio of the HD NSFET Cell (48:48), resulting in improved RSNM. However, it should be noted that the SL-FinFET exhibits weaker gate control capability and more pronounced DIBL effect compared to the NSFET. As a consequence, the driving ability improvement provided by the SL-FinFET diminishes gradually at lower operating voltages, ultimately causing the disappearance of the RSNM improvement in the HD HI-R Cell under such conditions. The WSNM exhibits an inverse trend compared to RSNM, as shown in Fig. 7(c). In the HD HI-R Cell, there is a slight reduction in WSNM compared to the HD NSFET Cell. This trade-off involves sacrificing WSNM to enhance RSNM, aligning both parameters to reach a closer level, which balances the conflict between read and write stability and leads to a considerable improvement in the overall disturb margin. In contrast, the HD HI-S Cell benefits from a larger AX width, resulting in a substantial enhancement of WSNM. At the operating voltage of 0.7 V, the HD HI-S Cell achieves a significant 29.6% improvement in WSNM compared to the HD NSFET Cell.

Fig. 8 shows the RT and WT characteristics of the HD SRAM Cells. At the operating voltage of 0.7 V, the HD HI-S Cell demonstrates a 21.6% reduction in RT and a 28.6% reduction in WT compared to the HD NSFET Cell. This notable improvement in speed is primarily attributed to the implementation of p-type SL-FinFET, which accelerates the charging and discharging process of storage nodes and BLs

Authorized licensed use limited to: Tamkang Univ.. Downloaded on February 04,2026 at 07:12:59 UTC from IEEE Xplore. Restrictions apply.

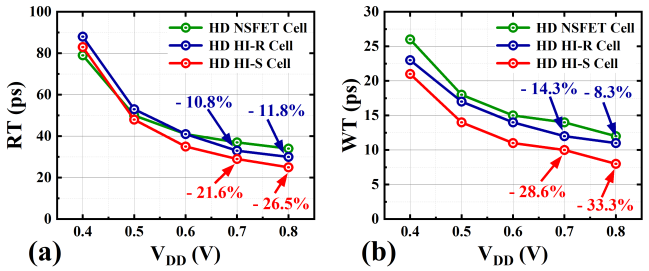


Fig. 8. Variation of (a) RT and (b) WT of HD 6T-SRAM cells with V_{DD}. The HI-S Cell shows great improvement in speed.

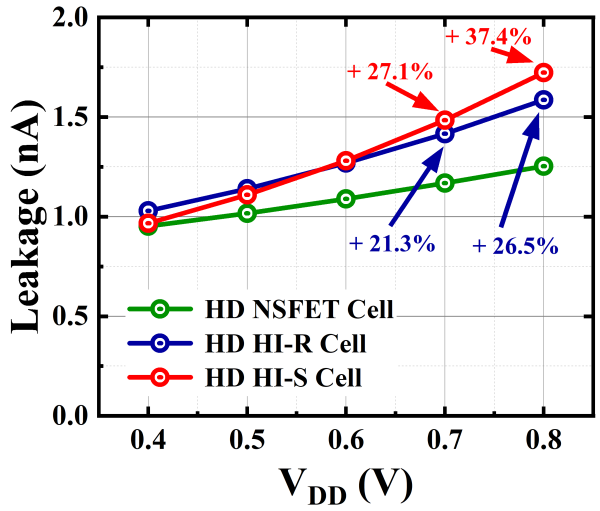


Fig. 9. Variation of leakage of HD 6T-SRAM cells with V_{DD}.

during write and read operations. On the other hand, the HD HI-R Cell exhibits a reduction in RT compared to the HD NSFET Cell due to the replacement of NSFETs by SL-FinFETs on the critical read path. Additionally, the use of p-type SL-FinFETs for the PU enhances the charging speed of the storage node during the writing operations, resulting in a reduced WT. The shortened RT and WT of HI-SRAM will enable them to operate at higher frequencies. However, it is important to note that the improvement in transient characteristic comes at the cost of higher power consumption, as shown in Fig. 9.

B. HP SRAM Optimization

Two distinct types of HP NSFET SRAM cells were designed by leveraging the flexibility of the NSFET width. The first type is the HP NSFET Cell for high-speed operations (HP-S NSFET Cell), where the AX and PD widths increase synchronously. The second type is HP NSFET Cell for robustness (HP-R NSFET Cell), where only the PD width is increased. The HP HI-SRAM cells achieve performance improvements by increasing the number of quantized Fin (see Fig. 6).

Fig. 10(a) shows the RT of the HP SRAM cells. With increasing AX and PD widths, the RT of the HP-S NSFET Cell decreases rapidly. When the cell height reaches 208 nm (with AX and PD widths of 21 nm), the RT reaches its minimum

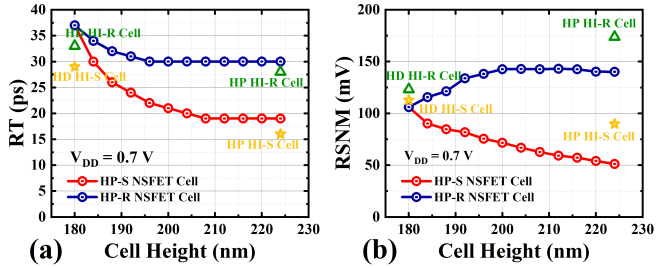


Fig. 10. Variation of (a) RT and (b) RSNM with cell height of HP 6T-SRAM cells at 0.7 V operating voltage.

value of 19 ps. However, further speed improvements are limited by the increased gate capacitance resulting from the continued increase in AX width. The charging time for the AX gate capacitance becomes the primary factor restricting further enhancements in speed. Regarding the HP-R NSFET Cell, the RT also slightly decreases due to the increased current on the critical read path. The HI-SRAM cells exhibit even better speed performance. The speed of the HD HI-S Cell is comparable to that of the HP-S NSFET Cell with a cell height of 186 nm. Moreover, the RT of the HP HI-S Cell continues to decrease by 15.8% below the limit value of the HP-S NSFET Cell. Although the speed of the HI-R Cell is inferior to that of the HP-S NSFET Cell, it still surpasses the speed of the HP-R NSFET Cell. The adoption of SL-FinFET, with its larger current brings a noteworthy breakthrough in speed compared to traditional NSFET SRAM cells.

Fig. 10(b) shows the RSNM of the HP SRAM cells. The RSNM of the HP-R NSFET Cell increases as the PD ratio grows due to the wider PD width. When the cell height reaches 200 nm (with a PD width of 17 nm), the HP-R NSFET Cell achieves a maximum RSNM of 143 mV. The HI-R Cells exhibit even better stability performance. The RSNM of the HD HI-R Cell is comparable to that of the HP-R NSFET Cell with a cell height of 188 nm. Furthermore, the RSNM of the HP HI-R Cell continues to increase by 21.6% beyond the limit value of the HP-R NSFET Cell. The hybrid integration of SL-FinFET and NSFET brings a larger transistor ratio and results in a notable breakthrough improvement in RSNM. On the other hand, the HP HI-S Cell and the HP-S NSFET Cell utilize larger AX widths but sacrifice the transistor ratio design, leading to a decrease in RSNM. The larger PU current of the HP HI-S Cell makes it more challenging to rewrite stored information, resulting in a smaller reduction in RSNM compared to the HP-S NSFET Cell.

C. SiGe Thickness Optimization in the Si/SiGe Epitaxy Process

The hybrid integration scheme enables the adjustment of the transistor ratio in the height dimension, overcoming the limitations of traditional method that relies on width design and resulting in a significant reduction in circuit area. This adjustment is achieved by modifying the SiGe thickness in the Si/SiGe epitaxy process. In SL-FinFET, the effective channel width (W_{eff}) is primarily determined by the height of the Fin, allowing changes in SiGe thickness to alter the W_{eff}. However, for NSFET, the W_{eff} is determined by the perimeter

Authorized licensed use limited to: Tamkang Univ.. Downloaded on February 04,2026 at 07:12:59 UTC from IEEE Xplore. Restrictions apply.

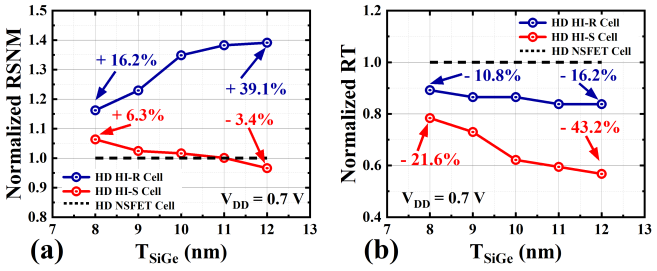


Fig. 11. (a) Normalized RSNM and (b) normalized RT versus T_{SiGe} of HD 6T-SRAM cells at 0.7 V operating voltage.

of Si nanosheets, so the variation in SiGe thickness does not affect the W_{eff} of NSFET. In other words, increasing the SiGe thickness during the Si/SiGe epitaxy process on one wafer may increase the ratio of SL-FinFET to NSFET without increasing the transistor occupied area, even though both kinds of the devices on the wafer have the same channel height. Fig. 11 shows the impact of SiGe thickness on the performance of HD SRAM cells. Since the SiGe thickness has little effect on the NSFET current, the performance of HD NSFET SRAM cells remains essentially unchanged. For the HD HI-R Cell, increasing the SiGe thickness from 8 to 12 nm leads to an improvement in RSNM from 16.2% to 39.1%. The PU ratio of the HD HI-R Cell continues to increase with SiGe thickness, further enhancing RSNM. Regarding the HD HI-S Cell, increasing the SiGe thickness from 8 to 12 nm results in a reduction in RT from 21.6% to 43.2%. The current of the SL-FinFET steadily increases with SiGe thickness, leading to further RT reduction in the HD HI-S Cell. Overall, increased SiGe thickness enhances the advantages of HI-SRAM cells.

VI. Conclusion

In summary, this article presents an innovative SRAM bitcell scheme that hybrid integrates both NSFET and SL-FinFET in one cell. This hybrid integration method is fully compatible with mainstream NSFET manufacturing processes, achieved through the precise formation of selective nanosheet channels. By harnessing the inherent transistor ratios provided by Si NSFET and SiGe SL-FinFET, this revolutionary scheme enables considerable performance enhancements while reducing the area. Compared to the HD NSFET SRAM cell, the HD HI-R Cell achieves a noteworthy 16.2% improvement in RSNM, while the HD HI-S Cell demonstrates notable reductions of 21.6% in RT and 28.6% in WT. The HP HI-S Cells even push the performance of SRAM cells to approach the theoretical limits set by regular NSFETs. Moreover, the hybrid integration scheme offers a promising pathway to optimize the transistor ratio in SRAM cells by adjusting the SiGe thickness in the transistors, thereby further enhancing their performance. The proposed technology provides one promising approach to scaling SRAM with higher performance and density for 3 nm and beyond nodes.

References

[1] H. Fujiwara et al., "A 7 nm 2.1 GHz dual-port SRAM with WL-RC optimization and dummy-read-recovery circuitry to mitigate read-disturb-write issue," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, San Francisco, CA, USA, Feb. 2019, pp. 390–392, doi: 10.1109/ISSCC.2019.8662415.

[2] J. Chang et al., "A 5 nm 135 Mb SRAM in EUV and high-mobility-channel FinFET technology with metal coupling and charge-sharing write-assist circuitry schemes for high-density and low-VMIN applications," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, Apr. 2020, pp. 238–240.

[3] S. Barraud, "Performance and design considerations for gate-all-around stacked-nanowires FETs," in *IEDM Tech. Dig.*, Dec. 2017, pp. 29.2.1–29.2.4, doi: 10.1109/IEDM.2017.8268473.

[4] D. Yakimets et al., "Power aware FinFET and lateral nanosheet FET targeting for 3 nm CMOS technology," in *IEDM Tech. Dig.*, Dec. 2017, pp. 20.4.1–20.4.4, doi: 10.1109/IEDM.2017.8268429.

[5] M. Garcia Bardon et al., "Power-performance trade-offs for lateral NanoSheets on ultra-scaled standard cells," in *Proc. IEEE Symp. VLSI Technol.*, Jun. 2018, pp. 143–144, doi: 10.1109/VLSIT.2018.8510633.

[6] U. K. Das and T. K. Bhattacharyya, "Opportunities in device scaling for 3-nm node and beyond: FinFET versus GAA-FET versus UFET," *IEEE Trans. Electron Devices*, vol. 67, no. 6, pp. 2633–2638, Jun. 2020, doi: 10.1109/TED.2020.2987139.

[7] F. M. Bufler, R. Ritzenthaler, H. Mertens, G. Eneman, A. Mocuta, and N. Horiguchi, "Performance comparison of *n*–type Si nanowires, nanosheets, and FinFETs by MC device simulation," *IEEE Electron Device Lett.*, vol. 39, no. 11, pp. 1628–1631, Nov. 2018, doi: 10.1109/LED.2018.2868379.

[8] Q. Zhang et al., "Optimization of structure and electrical characteristics for four-layer vertically-stacked horizontal gate-all-around Si nanosheets devices," *Nanomaterials*, vol. 11, no. 3, p. 646, Mar. 2021, doi: 10.3390/nano11030646.

[9] N. Loubet et al., "Stacked nanosheet gate-all-around transistor to enable scaling beyond FinFET," in *Proc. Symp. VLSI Technol.*, Kyoto, Japan, Jun. 2017, pp. T230–T231, doi: 10.23919/VLSIT.2017.7998183.

[10] Q. Huo et al., "Physics-based device-circuit cooptimization scheme for 7-nm technology node SRAM design and beyond," *IEEE Trans. Electron Devices*, vol. 67, no. 3, pp. 907–914, Mar. 2020, doi: 10.1109/TED.2020.2964610.

[11] T. Song et al., "A 3 nm gate-all-around SRAM featuring an adaptive dual-BL and an adaptive cell-power assist circuit," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, vol. 64, San Francisco, CA, USA, Feb. 2021, pp. 338–340, doi: 10.1109/ISSCC42613.2021.9365988.

[12] G. Bae et al., "3 nm GAA technology featuring multi-bridge-channel FET for low power and high performance applications," in *IEDM Tech. Dig.*, San Francisco, CA, USA, Dec. 2018, pp. 28.7.1–28.7.4, doi: 10.1109/IEDM.2018.8614629.

[13] T. H. Bao et al., "Circuit and process co-design with vertical gate-all-around nanowire FET technology to extend CMOS scaling for 5nm and beyond technologies," in *Proc. 44th Eur. Solid State Device Res. Conf. (ESSDERC)*, Venice Lido, Italy, Sep. 2014, pp. 102–105, doi: 10.1109/ESSDERC.2014.6948768.

[14] A. Veloso et al., "Challenges and opportunities of vertical FET devices using 3D circuit design layouts," in *Proc. IEEE SOI-3D-Subthreshold Microelectron. Technol. Unified Conf. (S3S)*, Burlingame, CA, USA, Oct. 2016, pp. 1–3, doi: 10.1109/S3S.2016.7804409.

[15] T. Huynh-Bao et al., "Design technology co-optimization for enabling 5nm gate-all-around nanowire 6T SRAM," in *Proc. Int. Conf. IC Design Technol. (ICICDT)*, Leuven, Belgium, Jun. 2015, pp. 1–4, doi: 10.1109/ICICDT.2015.7165874.

[16] T. Huynh-Bao et al., "A comprehensive benchmark and optimization of 5-nm lateral and vertical GAA 6T-SRAMs," *IEEE Trans. Electron Devices*, vol. 63, no. 2, pp. 643–651, Feb. 2016, doi: 10.1109/TED.2015.2504729.

[17] V. Vashishtha and L. T. Clark, "Comparing bulk-Si FinFET and gate-all-around FETs for the 5 nm technology node," *Microelectron. J.*, vol. 107, Jan. 2021, Art. no. 104942, doi: 10.1016/j.mejo.2020.104942.

[18] J. Qian, S. Pullela, and L. Pillage, "Modeling the 'Effective capacitance' for the RC interconnect of CMOS gates," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 13, no. 12, pp. 1526–1535, Dec. 1994, doi: 10.1109/43.331409.

[19] T. Yamashita et al., "A novel ALD SiBCN low-k spacer for parasitic capacitance reduction in FinFETs," in *Proc. Symp. VLSI Technol.*, Kyoto, Japan, Jun. 2015, pp. T154–T155, doi: 10.1109/VLSIT.2015.7223659.

[20] H. Mertens et al., "Vertically stacked gate-all-around Si nanowire transistors: Key process optimizations and ring oscillator demonstration," in *IEDM Tech. Dig.*, San Francisco, CA, USA, Dec. 2017, pp. 37.4.1–37.4.4, doi: 10.1109/IEDM.2017.8268511.

Authorized licensed use limited to: Tamkang Univ.. Downloaded on February 04,2026 at 07:12:59 UTC from IEEE Xplore. Restrictions apply.

[21] K. Cheng et al., "Improved air spacer co-integrated with self-aligned contact (SAC) and contact over active gate (COAG) for highly scaled CMOS technology," in *Proc. IEEE Symp. VLSI Technol.*, Honolulu, HI, USA, Jun. 2020, pp. 1–2, doi: 10.1109/VLSITechnology18217.2020.9265096.

[22] S. J. Kang, J. H. Kim, Y. S. Song, S. Go, and S. Kim, "Investigation of self-heating effects in vertically stacked GAA MOSFET with wrap-around contact," *IEEE Trans. Electron Devices*, vol. 69, no. 3, pp. 910–914, Mar. 2022, doi: 10.1109/TED.2022.3140283.

[23] L. Gaben et al., "(Invited) evaluation of stacked nanowires transistors for CMOS: Performance and technology opportunities," *ECS Trans.*, vol. 72, no. 4, pp. 43–54, May 2016, doi: 10.1149/07204.0043ecst.

[24] M. K. Gupta et al., "A comprehensive study of nanosheet and forksheet SRAM for beyond n5 node," *IEEE Trans. Electron Devices*, vol. 68, no. 8, pp. 3819–3825, Aug. 2021, doi: 10.1109/TED.2021.3088392.

[25] G. Hellings et al., "Si/SiGe superlattice I/O finFETs in a vertically-stacked gate-all-around horizontal nanowire technology," in *Proc. IEEE Symp. VLSI Technol.*, Honolulu, HI, USA, Jun. 2018, pp. 85–86, doi: 10.1109/VLSIT.2018.8510654.

[26] W.-C. Chen et al., "External I/O interfaces in sub-5nm GAA NS technology and STCO scaling options," in *Proc. Symp. VLSI Technol.*, Kyoto, Japan, Jun. 2021, pp. 1–2.

[27] L.-J. Liu et al., "High-performance pMOSFETs with high-k gate dielectric and dislocation-free epitaxial Si/Ge super-lattice channel," in *Proc. IEEE Silicon Nanoelectronics Workshop (SNW)*, Honolulu, HI, USA, Jun. 2012, pp. 1–2, doi: 10.1109/SNW.2012.6243324.

[28] L.-J. Liu, K.-S. Chang-Liao, Y.-C. Jian, J.-W. Cheng, T.-K. Wang, and M.-J. Tsai, "Performance enhancement on p-channel charge-trapping flash memory device with epitaxial Si/Ge super-lattice channel," *IEEE Electron Device Lett.*, vol. 34, no. 5, pp. 587–589, May 2013, doi: 10.1109/LED.2013.2250476.

[29] S. Bhattacharya, A. Kundu, D. Chakraborty, A. Sarkar, and M. Mukherjee, "Strain modulated asymmetrical Si/SiGe superlattice p+-i-n+ switches for MMW low-loss secure communication systems," *IEEE Trans. Device Mater. Rel.*, vol. 23, no. 1, pp. 14–26, Mar. 2023, doi: 10.1109/TDMR.2022.3224444.

[30] Y.-L. Li et al., "Improved electrical characteristics of bulk FinFETs with SiGe super-lattice-like buried channel," *IEEE Electron Device Lett.*, vol. 40, no. 2, pp. 181–184, Feb. 2019, doi: 10.1109/LED.2018.2890535.

[31] M. U. Mohammed, A. Nizam, L. Ali, and M. H. Chowdhury, "FinFET based SRAMs in sub-10nm domain," *Microelectron. J.*, vol. 114, Aug. 2021, Art. no. 105116, doi: 10.1016/j.mejo.2021.105116.

[32] J. Yao et al., "Record 7(N)+7(P) multiple VTs demonstration on GAA Si nanosheet n/pFETs using WFM-less direct interfacial La/Al-dipole technique," in *IEDM Tech. Dig.*, San Francisco, CA, USA, Dec. 2022, pp. 34.2.1–34.2.4, doi: 10.1109/IEDM45625.2022.10019361.

[33] (2022). *International Roadmap for Devices and Systems More Moore White Paper*. [Online]. Available: https://irds.ieee.org/images/files/pdf/2022/2022IRDS_MM.pdf

[34] L. Cao et al., "Novel channel-first fishbone FETs with symmetrical threshold voltages and balanced driving currents using single work function metal process," *IEEE Trans. Electron Devices*, vol. 69, no. 11, pp. 5971–5977, Nov. 2022, doi: 10.1109/TED.2022.3206179.

[35] S.-H. Lee et al., "V_{th} variation and strain control of high Ge% thin SiGe channels by millisecond anneal realizing high performance pMOSFET beyond 16nm node," in *Proc. Symp. VLSI Technol.*, Jun. 2009, pp. 74–75.

[36] H. Xu et al., "Physical insights of Si-core-SiGe-shell gate-all-around nanosheet pFET for 3 nm technology node," *IEEE Trans. Electron Devices*, vol. 70, no. 6, pp. 3365–3371, Jun. 2023, doi: 10.1109/TED.2023.3268156.

[37] C. H. Lee et al., "Toward high performance SiGe channel CMOS: Design of high electron mobility in SiGe nFinFETs outperforming Si," in *IEDM Tech. Dig.*, San Francisco, CA, USA, Dec. 2018, pp. 35.1.1–35.1.4, doi: 10.1109/IEDM.2018.8614581.

[38] H. Okamura et al., "A 1 ns, 1 W, 2.5 V, 32 kb NTL-CMOS SRAM macro using a memory cell with p-channel access transistors," in *Proc. ISSCC*, San Francisco, CA, USA, 1995, pp. 306–307, doi: 10.1109/isscc.1995.535567.

[39] Y. Luo et al., "Layout optimization of complementary FET 6T-SRAM cell based on a universal methodology using sensitivity with respect to parasitic-and-values," *IEEE Trans. Electron Devices*, vol. 69, no. 11, pp. 6095–6101, Nov. 2022, doi: 10.1109/TED.2022.3207972.

Authorized licensed use limited to: Tamkang Univ.. Downloaded on February 04,2026 at 07:12:59 UTC from IEEE Xplore. Restrictions apply.